Kubernetes — Interview Ready Answers

Comprehensive answers with examples for all 45 Kubernetes questions

45 Questions • Pods • Deployments • StatefulSets • HPA • Helm • GitOps • Service Mesh • Security • Networking • Storage

Q1. How do you expose a Kubernetes application to external traffic?

Answer

Kubernetes offers multiple mechanisms to expose applications externally. The right choice depends on your protocol (HTTP vs TCP), whether you need routing logic, and your cloud provider.

1. NodePort — Direct Node Access

Exposes the Service on a static port on every node's IP. Traffic arriving at <NodeIP>:<NodePort> is forwarded to the Service. Rarely used in production — no load balancing across nodes, exposes the node's IP directly.

```
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080    # Must be 30000-32767
```

2. LoadBalancer — Cloud Native (Production Standard)

Creates a cloud provider load balancer (AWS ALB/NLB, Azure LB, GCP LB) that routes external traffic to the Service. The most common method for production services on managed Kubernetes (EKS, AKS, GKE).

```
apiVersion: v1
kind: Service
metadata:
  name: my-app
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: nlb
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 443
      targetPort: 8443
```

3. Ingress — HTTP/HTTPS Routing (Most Feature-Rich)

An Ingress resource defines routing rules (path-based, host-based). An Ingress Controller (NGINX, AWS Load Balancer Controller, Traefik) watches Ingress resources and configures the actual load balancer. One load balancer serves many services — most cost-effective for HTTP traffic.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
  - hosts: [api.myapp.com]
    secretName: api-tls
  rules:
  - host: api.myapp.com
    http:
      paths:
      - path: /v1
        pathType: Prefix
        backend:
          service:
            name: api-v1
            port: { number: 80 }
```

```
      - path: /v2
        pathType: Prefix
        backend:
          service:
            name: api-v2
            port: { number: 80 }
```

4. ExternalName — DNS Alias

Maps a Service to an external DNS name. Used to redirect cluster traffic to external services without changing application code:

```
kind: Service
spec:
  type: ExternalName
  externalName: my-db.us-east-1.rds.amazonaws.com
```

Comparison Table

Type	Use Case	Load Balancing
ClusterIP	Internal only	In-cluster
NodePort	Dev/test direct access	Basic
LoadBalancer	TCP/UDP production services	Cloud LB
Ingress	HTTP/HTTPS multi-service	Advanced routing
ExternalName	DNS alias to external service	None

> *Tip: For HTTP workloads, always use Ingress over multiple LoadBalancer services. Each LoadBalancer service provisions a separate cloud load balancer — expensive at scale. One Ingress Controller handles all HTTP routing from a single load balancer.*

Q2. What is the difference between Deployment and StatefulSet in Kubernetes?

Answer

Deployments and StatefulSets are both controllers that manage pods, but they serve fundamentally different application types. Choosing the wrong one causes data loss, identity conflicts, and startup failures.

Deployment — For Stateless Applications

A Deployment manages a set of identical, interchangeable pods. Pods have no stable identity — they can be rescheduled on any node, their names change on restart, and they don't retain state across restarts. All pods share the same volume mounts (if any) or are ephemeral.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  replicas: 3
  selector:
    matchLabels: { app: api-server }
  template:
    spec:
      containers:
      - name: api
        image: myapp:v1.2
        # Pods are interchangeable — api-server-abc123 or api-server-xyz456
        # Any pod can handle any request
```

StatefulSet — For Stateful Applications

A StatefulSet gives each pod a stable, predictable identity: a sequential ordinal name (pod-0, pod-1, pod-2), a stable network hostname, and a dedicated PersistentVolumeClaim per pod. Pods are created in order (0 first, then 1, then 2) and deleted in reverse order.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless
  replicas: 3
  selector:
```

```
    matchLabels: { app: postgres }
  template:
    spec:
      containers:
      - name: postgres
        image: postgres:15
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
      resources:
        requests:
          storage: 50Gi

# Pods get stable names: postgres-0, postgres-1, postgres-2
# Each has its own PVC: data-postgres-0, data-postgres-1, data-postgres-2
# DNS: postgres-0.postgres-headless.default.svc.cluster.local
```

Key Differences

Aspect	Deployment
Pod identity	Random (api-5d7f8b-xyz)
DNS hostname	No stable hostname
Storage	Shared or ephemeral
Scaling order	All pods start simultaneously
Update strategy	Rolling (any pod first)
Pod rescheduling	Reschedules anywhere, new name
Use cases	Web servers, APIs, workers
Headless Service	Optional

Headless Service — Required for StatefulSets

StatefulSets require a Headless Service (clusterIP: None) to provide stable DNS for each pod:

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None    # Headless — no load balancing
  selector:
    app: postgres
  ports:
  - port: 5432
```

With this, each pod gets a DNS entry: postgres-0.postgres-headless.default.svc.cluster.local. Cluster members can connect to specific replicas by name — essential for database primary/replica topologies.

> *Tip: Run stateful applications (databases, message queues) as StatefulSets in Kubernetes only if the operational complexity is justified. In many cases, managed cloud services (RDS, ElastiCache, MSK) are simpler and more reliable than managing stateful workloads in Kubernetes.*

Q3. What is a ConfigMap, and how is it different from a Secret?

Answer

ConfigMaps and Secrets both decouple configuration from container images, but they differ in their intended use and how Kubernetes handles their data.

ConfigMap — Non-Sensitive Configuration

A ConfigMap stores non-sensitive key-value pairs or configuration files. Values are stored as plaintext in etcd and are visible to anyone with read access to the ConfigMap resource.

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: warn
  MAX_CONNECTIONS: '100'
  config.yaml: |
    server:
      port: 8080
      timeout: 30s
    database:
      pool_size: 10
```

Secret — Sensitive Configuration

A Secret stores sensitive data. Values are base64-encoded (NOT encrypted by default). When etcd encryption at rest is enabled, secrets are encrypted. Kubernetes applies additional access controls — RBAC can restrict Secret access more granularly than ConfigMap access.

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:             # Use stringData for plaintext — auto-encoded
  DB_PASSWORD: 's3cr3tP@ss'
  DB_USERNAME: 'appuser'
  API_KEY: 'sk-abc123'
```

Consuming ConfigMaps and Secrets in Pods

Both can be consumed as environment variables or volume mounts:

```
spec:
  containers:
  - name: app
    image: myapp:v1
    env:
    # ConfigMap as env vars
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
    # Secret as env var
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: DB_PASSWORD
    # Full ConfigMap as env vars
    envFrom:
    - configMapRef:
        name: app-config
    # Secret mounted as files
    volumeMounts:
    - name: secret-vol
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: secret-vol
    secret:
      secretName: db-credentials
      defaultMode: 0400    # Read-only by owner only
```

Key Differences

Aspect	ConfigMap
Purpose	Non-sensitive config (env, flags, files)
Storage encoding	Plaintext in etcd
Access control	Standard RBAC
Size limit	1MB per ConfigMap
Volume mount	Mounted as files or env vars
Auto-update in pods	Yes (volume mounts update

	within ~1min)

Important Limitations of Kubernetes Secrets

- Secrets are base64-encoded, NOT encrypted by default — anyone with kubectl get secret can decode them.
- Enable etcd encryption at rest on your cluster for true encryption.
- For production, use an external secret manager (AWS Secrets Manager, Vault) with External Secrets Operator or Secrets Store CSI Driver — do not rely on native Kubernetes Secrets alone for sensitive production data.

> *Tip: Never store secrets as environment variables if you can avoid it — they appear in kubectl describe pod output and process listings. Mount secrets as read-only files instead. Applications read from /etc/secrets/db-password, which is less exposed.*

Q4. How would you implement blue-green deployment in Kubernetes?

Answer

Blue-green deployment runs two identical environments simultaneously. Blue is the current production version; Green is the new version. Traffic switches instantly from Blue to Green when Green is verified — enabling zero-downtime releases with instant rollback.

Implementation Using Service Selector Switching

The simplest approach uses native Kubernetes Services. Switch traffic by updating the service's pod selector:

```
# Blue Deployment (currently live)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-blue
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
      version: blue
  template:
    metadata:
      labels:
        app: my-app
        version: blue
    spec:
      containers:
      - name: app
        image: myapp:v1.0

# Green Deployment (new version — not yet live)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-green
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
      version: green
  template:
    metadata:
      labels:
        app: my-app
        version: green
    spec:
      containers:
      - name: app
        image: myapp:v2.0    # New version

# Service — currently points to blue
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  selector:
    app: my-app
```

```
    version: blue    # Change this to 'green' for cutover
  ports:
  - port: 80
    targetPort: 8080
```

Cutover and Rollback

```
# Step 1: Deploy green (runs alongside blue, receives no traffic)
kubectl apply -f green-deployment.yaml

# Step 2: Verify green is healthy
kubectl rollout status deployment/my-app-green
kubectl get pods -l version=green
# Run smoke tests against green pods directly
kubectl port-forward deploy/my-app-green 8080:8080
curl localhost:8080/health

# Step 3: Switch traffic to green (instant — no downtime)
kubectl patch service my-app \
  -p '{"spec":{"selector":{"app":"my-app","version":"green"}}}'

# Step 4: Verify production traffic on green
kubectl get endpoints my-app  # Should show green pod IPs

# Step 5a: If green is healthy — clean up blue
kubectl delete deployment my-app-blue

# Step 5b: If green has issues — instant rollback to blue
kubectl patch service my-app \
  -p '{"spec":{"selector":{"app":"my-app","version":"blue"}}}'
```

Automating in CI/CD Pipeline

```
# GitHub Actions blue-green deploy step
- name: Blue-Green Deploy
  run: |
    # Deploy green
    kubectl set image deployment/my-app-green app=myapp:${{ github.sha }}
    kubectl rollout status deployment/my-app-green --timeout=120s

    # Automated smoke test
    GREEN_POD=$(kubectl get pod -l version=green -o jsonpath='{.items[0].metadata.name}')
    kubectl exec $GREEN_POD -- curl -sf localhost:8080/health

    # Cut over
    kubectl patch service my-app -p '{"spec":{"selector":{"version":"green"}}}'
    echo 'Traffic switched to green'
```

Advanced: Blue-Green with Helm

```
# values-blue.yaml
activeSlot: blue
image.tag: v1.0

# values-green.yaml
activeSlot: green
image.tag: v2.0

helm upgrade --install my-app ./chart -f values-green.yaml
```

> *Tip: Keep the inactive environment (Blue after cutover to Green) running for at least 30 minutes before deleting. This maintains an instant rollback option during the observation window. Deleting too quickly eliminates your safety net.*

Q5. How do you implement network policies to restrict pod-to-pod communication in Kubernetes?

Answer

By default, Kubernetes allows all pods to communicate with all other pods across all namespaces — this is the 'allow all' default that violates the principle of least privilege. NetworkPolicy resources define rules to restrict this traffic. A CNI plugin that supports NetworkPolicy (Calico, Cilium, Weave) must be installed — standard kubenet does not enforce NetworkPolicy.

Default Deny — Foundation of Zero-Trust Networking

Start by denying all ingress and egress traffic to all pods in a namespace, then add allow rules only for required communication:

```
# Default deny all ingress and egress in 'payments' namespace
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: payments
spec:
  podSelector: {}    # Applies to ALL pods in namespace
  policyTypes:
  - Ingress
  - Egress
  # No ingress/egress rules = deny everything
```

Allow Specific Pod-to-Pod Communication

Allow the API gateway to reach the payment service on port 8080 only:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-to-payments
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service    # Policy applies to payment-service pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: api-gateway         # Only from api-gateway namespace
      podSelector:
        matchLabels:
          app: api-gateway          # Only from api-gateway pods
    ports:
    - protocol: TCP
      port: 8080
```

Allow Egress to Database Only

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: payments-egress
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          name: databases
      podSelector:
        matchLabels:
          app: postgres
    ports:
    - protocol: TCP
      port: 5432
  # Allow DNS resolution (always needed)
  - to:
    - namespaceSelector: {}
    ports:
    - protocol: UDP
      port: 53
```

Allow Monitoring to Scrape All Pods

```
# Allow Prometheus in monitoring namespace to scrape metrics
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-prometheus-scrape
  namespace: payments
spec:
  podSelector: {}
  policyTypes:
```

```
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: monitoring
      podSelector:
        matchLabels:
          app: prometheus
    ports:
    - port: 9090
      protocol: TCP
```

Testing Network Policies

```
# Test connectivity — should fail with default-deny
kubectl exec -it test-pod -n api-gateway -- \
  nc -zv payment-service.payments.svc.cluster.local 8080
# Expected: Connection refused (policy blocks it)

# After applying allow policy — should succeed
kubectl exec -it api-pod -n api-gateway -- \
  nc -zv payment-service.payments.svc.cluster.local 8080
# Expected: Connection succeeded
```

Cilium — Advanced Network Policies

Cilium extends NetworkPolicy with L7 (application-layer) policies and identity-based security using eBPF. It can enforce policies based on HTTP methods, paths, gRPC services, and DNS names — not just IP/port:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
spec:
  endpointSelector:
    matchLabels:
      app: payment-service
  ingress:
  - fromEndpoints:
    - matchLabels:
        app: api-gateway
    toPorts:
    - ports:
      - port: '8080'
      rules:
        http:
        - method: POST
          path: /api/v1/payment    # Allow only POST to payment endpoint
```

Tip: Always include a DNS egress allow rule (UDP port 53) when applying egress NetworkPolicies. Forgetting this blocks all DNS resolution and breaks the service even if all other traffic is correctly allowed.

Q6. A critical production Kubernetes cluster is experiencing multiple issues. Pods are stuck in ImagePullBackOff, some pods are being evicted, and users are reporting 503 errors. What troubleshooting process will you follow?

Answer

This is a multi-symptom incident requiring parallel investigation. Triage each symptom, identify root causes, and remediate in priority order — restoring availability first, then fixing root causes.

Step 1 — Immediate Triage: Get the Full Picture

```
# Overview of cluster health
kubectl get nodes                          # Node status
kubectl get pods --all-namespaces | grep -v Running   # Non-running pods
kubectl top nodes                          # Node resource pressure
kubectl top pods --all-namespaces          # Pod resource usage
kubectl get events --all-namespaces --sort-by=.lastTimestamp | tail -30
```

Problem 1 — ImagePullBackOff

Pods cannot pull their container image. Common causes:

- Image tag does not exist in the registry.
- Registry credentials are missing, expired, or wrong.
- The image registry is unreachable from the cluster.
- Private registry requires imagePullSecret that isn't configured.

```
# Get exact error
kubectl describe pod <pod-name> | grep -A10 'Events'
# Look for: 'Failed to pull image' or '401 Unauthorized' or '404 Not Found'

# Fix 1: Wrong tag — update deployment to correct tag
kubectl set image deployment/my-app app=myapp:v1.2.3

# Fix 2: Missing imagePullSecret
kubectl create secret docker-registry ecr-secret \
  --docker-server=123456.dkr.ecr.us-east-1.amazonaws.com \
  --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password)

# Add imagePullSecret to deployment
kubectl patch deployment my-app -p \
  '{"spec":{"template":{"spec":{"imagePullSecrets":[{"name":"ecr-secret"}]}}}}'

# Fix 3: For EKS — ensure node IAM role has ECR pull permissions
# Attach AmazonEC2ContainerRegistryReadOnly policy to node group role
```

Problem 2 — Pod Eviction

Kubelet evicts pods when nodes are under resource pressure (disk, memory). Eviction messages identify the cause:

```
kubectl get events | grep -i evict
kubectl describe node <node-name> | grep -A10 'Conditions'
# Look for: MemoryPressure, DiskPressure, PIDPressure True

# Immediate relief — scale up nodes or add spot nodes
kubectl scale deployment my-app --replicas=2   # Reduce replica count temporarily

# Check what's consuming memory
kubectl top pods --all-namespaces --sort-by=memory | head -20

# Check disk on nodes
kubectl debug node/<node> -it --image=busybox -- df -h

# Root fixes:
# - Clean up unused images: crictl rmi --prune
# - Add resource limits to pods without them
# - Increase node size or add nodes
# - Clean up large logs: journalctl --vacuum-size=500M
```

Problem 3 — 503 Errors

503 from the load balancer means no healthy backends. Work backwards from the LB to the pods:

```
# Check Service endpoints — are any pods registered?
kubectl get endpoints my-app-service
# Empty endpoints = no pods matching selector or all pods failing health checks

# Check pod readiness
kubectl get pods -l app=my-app
# Look for: 0/1 READY — pods are running but not ready

# Check readiness probe failures
kubectl describe pod <pod> | grep -A5 'Readiness'
kubectl logs <pod> --previous   # Logs from crashed container

# Test directly (bypass LB and service)
kubectl port-forward pod/<pod-name> 8080:8080
curl localhost:8080/health
```

Root Cause Analysis and Prevention

Issue	Root Cause
ImagePullBackOff	Image tag doesn't exist; missing ECR auth
Pod eviction	No resource limits; disk full from logs
503 errors	Pods not ready; health check fails

Future Prevention

- Resource quotas and LimitRanges — prevent pods without limits from consuming entire nodes.

- PodDisruptionBudgets — ensure minimum pods stay available during disruptions.
- ImagePullPolicy: IfNotPresent with immutable tags — reduce registry dependency.
- Node autoscaling (Karpenter/Cluster Autoscaler) — add capacity before eviction pressure builds.
- Monitoring and alerting on: node pressure conditions, pod eviction events, endpoint count dropping.

> *Tip: Create a documented runbook for each of these failure modes. When an incident occurs at 3 AM, a clear runbook dramatically reduces MTTR and prevents engineers from making hasty mistakes under pressure.*

Q7. How would you handle CPU throttling in a busy AKS/EKS cluster under payment traffic load?

Answer

CPU throttling occurs when a container uses more CPU than its limit allows. The Linux kernel's CFS (Completely Fair Scheduler) throttles the container, causing increased latency and timeouts — particularly damaging for payment workloads where response time directly impacts conversion rates.

Understanding CPU Throttling

Kubernetes CPU limits work through Linux cgroup quota enforcement. When a container is limited to 0.5 CPU, the kernel allows it 50ms of CPU time per 100ms period. If it tries to use more, it's paused (throttled) for the remainder of the period. This causes irregular, bursty latency — a 50ms request can take 150ms due to throttling pauses.

```
# Detect throttling — check container CPU throttling metrics
kubectl exec -it <pod> -- cat /sys/fs/cgroup/cpu/cpu.stat
# nr_throttled: 1234    # Number of throttled periods
# throttled_time: 5000000000  # Total ns throttled (5 seconds!)

# In Prometheus:
rate(container_cpu_cfs_throttled_periods_total[5m])
  / rate(container_cpu_cfs_periods_total[5m])
# > 0.25 means >25% of CPU periods are throttled — investigate
```

Step 1 — Right-Size CPU Requests and Limits

The most common cause: CPU limits set too low relative to actual usage. Profile actual CPU usage first, then set limits appropriately:

```
# Check actual CPU usage vs limits
kubectl top pods -l app=payment-service

# View resource configuration
kubectl get pod <payment-pod> -o jsonpath=
  '{.spec.containers[0].resources}'

# Increase CPU limit if consistently throttled
kubectl set resources deployment payment-service \
  --requests=cpu=500m,memory=512Mi \
  --limits=cpu=2000m,memory=1Gi
```

Step 2 — Remove CPU Limits (Controversial but Valid)

For latency-sensitive payment workloads, consider removing CPU limits entirely. The container can burst above its request during traffic spikes without throttling. Use requests (for scheduling) but no limits (for burstability):

```
resources:
  requests:
    cpu: 500m        # Used for scheduling — guaranteed
    memory: 512Mi
  # No limits — pod can burst to use spare node CPU
  # Safe on dedicated node groups with appropriate node sizing
```

> *Note: Removing CPU limits requires careful node sizing and monitoring. Without limits, a single pod can starve other pods on the same node. Use this only on dedicated node groups with sufficient headroom.*

Step 3 — HPA with Custom Metrics

Scale out horizontally before throttling becomes a problem. For payment traffic, scale on request rate or queue depth — not just CPU:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payment-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
```

```
  minReplicas: 5
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60   # Scale at 60% — before throttling
  - type: External
    external:
      metric:
        name: payment_requests_per_second
      target:
        type: AverageValue
        averageValue: '100'
```

Step 4 — Node Sizing and Dedicated Node Groups

For payment services, use dedicated node groups with larger instance types that provide more CPU headroom per pod:

```
# EKS — dedicated payment node group
nodeGroups:
  - name: payment-nodes
    instanceType: m5.2xlarge    # 8 vCPUs per node
    taints:
    - key: dedicated
      value: payment
      effect: NoSchedule

# Payment pods tolerate the taint — only payment pods scheduled here
tolerations:
- key: dedicated
  value: payment
  effect: NoSchedule
```

Step 5 — Vertical Pod Autoscaler (VPA)

VPA automatically recommends or applies right-sized CPU and memory requests based on historical usage:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: payment-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
  updatePolicy:
    updateMode: 'Off'      # Recommendation mode — don't auto-apply in prod

# Check VPA recommendations
kubectl describe vpa payment-vpa
# Lower Bound: 250m    Upper Bound: 1500m   Target: 750m
```

> *Tip: For payment services, target <5% CPU throttling rate. Monitor container_cpu_cfs_throttled_periods_total in Prometheus and alert if throttling exceeds 10%. A burst of payment requests that causes throttling can cascade into timeouts, retries, and transaction failures — all impacting revenue.*

Q8. What are readiness gates in Kubernetes and when would you use them?

Answer

Readiness Gates extend the pod readiness mechanism beyond container health checks. They allow external systems to signal whether a pod is ready to serve traffic — enabling integration with cloud load balancers, service meshes, and custom deployment gates.

Standard Readiness vs Readiness Gates

Standard readiness: Kubernetes marks a pod ready when all containers pass their readiness probes. The pod is added to Service endpoints and receives traffic.

Problem: This doesn't account for external systems. A pod might pass its readiness probe but the cloud load balancer hasn't yet registered it, or the Istio sidecar hasn't finished initialising, or a smoke test hasn't verified it.

Readiness Gates: Additional conditions that must be true (set by external controllers) before Kubernetes considers the pod ready. A pod with readiness gates won't receive traffic until ALL readiness gate conditions are True.

Defining Readiness Gates

```
apiVersion: v1
kind: Pod
spec:
  readinessGates:
  - conditionType: 'target-health.elbv2.k8s.aws/my-tg-group'
  # Pod is ready ONLY when:
  # 1. Container readiness probe passes (standard)
  # 2. ALB Target Group reports this pod as healthy (readiness gate)
```

Use Case 1 — AWS Load Balancer Controller

The AWS Load Balancer Controller sets readiness gate conditions based on whether the ALB Target Group has registered and health-checked the pod. This prevents traffic from hitting the pod before the ALB is ready:

```
# The controller automatically sets this readiness gate
# when you annotate the service for ALB
apiVersion: v1
kind: Service
metadata:
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip

# Pods won't receive traffic until ALB reports them healthy
# Eliminates 503 errors during rolling deployments
```

Use Case 2 — Service Mesh Sidecar Readiness

Istio and Linkerd inject sidecar proxies into pods. If the application container starts before the sidecar is ready, requests fail. A readiness gate ensures both the app and sidecar are ready:

```
readinessGates:
- conditionType: 'istio.io/ready'
# Istio control plane sets this condition True
# when the Envoy sidecar has synced its configuration
```

Use Case 3 — Custom Deployment Gates

For zero-downtime deployments, implement custom readiness gates that require a smoke test to pass before the pod receives production traffic:

```
# Custom condition set by your deployment controller
readinessGates:
- conditionType: 'deployment.company.com/smoke-test-passed'

# Your smoke test controller:
# 1. Runs curl against the new pod
# 2. If smoke test passes, patches the pod condition to True
kubectl patch pod <pod-name> --subresource=status --type=json -p='
[{"op":"add","path":"/status/conditions/-","value":{
  "type":"deployment.company.com/smoke-test-passed",
  "status":"True"}}]'
```

Checking Readiness Gate Status

```
kubectl get pod <pod-name> -o yaml | grep -A10 conditions
# status:
#   conditions:
#   - type: Ready
#     status: 'False'    # Not yet ready
#   - type: target-health.elbv2.k8s.aws/my-tg-group
#     status: 'False'    # ALB hasn't registered yet
#   - type: ContainersReady
#     status: 'True'     # Container probe passed
```

> *Tip: Readiness Gates are essential for zero-downtime deployments with cloud load balancers. Without them, the rolling update replaces a pod before the cloud LB removes it from rotation, causing brief periods where the LB sends traffic to a pod that's already terminated.*

Q9. Explain how you'd debug an issue where the HPA is not scaling pods despite high CPU usage.

Answer

HPA (Horizontal Pod Autoscaler) not scaling despite high CPU is one of the most frustrating Kubernetes issues. It has many possible causes — systematic diagnosis is essential.

Step 1 — Check HPA Status

```
kubectl get hpa my-app-hpa
```

```
# NAME        REFERENCE        TARGETS         MINPODS  MAXPODS  REPLICAS
# my-app-hpa  Deployment/app   <unknown>/70%   2        20       2

kubectl describe hpa my-app-hpa
# Key sections to read:
# Conditions: ScalingActive=False — WHY is scaling disabled?
# Metrics: <unknown> — can't read CPU metrics
```

Cause 1 — Metrics Server Not Installed or Not Working

```
# HPA requires metrics-server to read CPU/memory
kubectl top pods   # If this fails, metrics-server is the problem

# Check metrics-server is running
kubectl get pods -n kube-system | grep metrics-server

# Install metrics-server if missing
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml

# If metrics-server is running but 'kubectl top' fails:
kubectl logs -n kube-system deployment/metrics-server
# Common error: x509 certificate — add --kubelet-insecure-tls to args
```

Cause 2 — Missing Resource Requests on Pods

HPA calculates CPU utilisation as: actual CPU / CPU request. If no CPU request is set, HPA cannot calculate utilisation and reports <unknown>:

```
# Check if pods have CPU requests
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].resources}'
# Bad output: {}  — no requests set

# Fix: Add CPU request to deployment
kubectl set resources deployment my-app \
  --requests=cpu=200m,memory=256Mi \
  --limits=cpu=1000m,memory=512Mi
```

Cause 3 — Already at Maximum Replicas

```
kubectl describe hpa | grep 'Max replicas'
# If current replicas = maxReplicas, HPA can't scale further
# Fix: Increase maxReplicas
kubectl patch hpa my-app-hpa -p '{"spec":{"maxReplicas":50}}'
```

Cause 4 — Stabilization Window (Scale-Out Too Slow)

HPA has a built-in scale-down stabilisation window (default 5 minutes) and scale-up cooldown (default 3 minutes). During a sudden spike, HPA may seem slow:

```
# Check current stabilization settings
kubectl get hpa my-app-hpa -o yaml | grep -A10 behavior

# Reduce scale-up stabilization for faster response
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30    # Default is 180s — too slow
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 300   # Keep 5min to prevent thrashing
```

Cause 5 — Custom Metrics Not Available

If scaling on custom Prometheus metrics, the custom metrics API adapter (Prometheus Adapter) may not be configured:

```
# Check custom metrics API
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | jq .
# If this fails: prometheus-adapter is not installed/configured

# Check available custom metrics
kubectl get --raw
'/apis/custom.metrics.k8s.io/v1beta1/namespaces/default/pods/*/http_requests_per_second'
```

Cause 6 — Pod Pending Due to Insufficient Node Resources

HPA triggers scaling but new pods stay Pending because there's no node capacity. HPA did its job — the cluster autoscaler isn't scaling the nodes:

```
kubectl get pods -l app=my-app
```

```
# Some pods: Pending — 'Insufficient cpu' or 'Insufficient memory'

# HPA scaled out but Cluster Autoscaler hasn't added nodes yet
kubectl describe pod <pending-pod> | grep -A5 Events
kubectl logs -n kube-system deployment/cluster-autoscaler | tail -20
```

Debugging Checklist

Check	Command
Metrics server	kubectl top pods
CPU requests set	kubectl describe pod
HPA conditions	kubectl describe hpa
Current vs max replicas	kubectl get hpa
Metric value visible	kubectl get hpa
Node capacity	kubectl describe node

> *Tip: Enable HPA event logging: kubectl describe hpa shows the last scaling decisions and reasons. The 'Conditions' section is the most diagnostic — FailedGetScale, FailedConvertHPA, and ScalingLimited conditions each tell you exactly why HPA isn't doing what you expect.*

Q10. How do you ensure blue-green or shadow deployment testing for critical services?

Answer

For critical services (payments, authentication), you need deployment strategies that validate the new version against real traffic before fully switching over. Blue-green provides instant rollback; shadow testing validates with zero user risk.

Blue-Green for Critical Services — Enhanced Process

For critical services, blue-green requires more rigorous validation before cutover than the basic approach. See Q4 for the basic mechanism. Enhanced additions for critical services:

- Synthetic transaction testing — Run automated end-to-end payment transactions against the green deployment before cutover:

```
# Run smoke tests against green before switching
GREEN_POD=$(kubectl get pod -l version=green -o jsonpath='{.items[0].metadata.name}')
kubectl exec $GREEN_POD -- python3 /tests/payment_smoke.py
# Test: create payment, capture, refund — full lifecycle
```

- Canary before full blue-green — Route 5% of traffic to green first, monitor for 30 minutes, then complete cutover:

```
# Use Istio or NGINX weighted routing for 5% canary first
# Only proceed to full switch if error rate < 0.1% and latency normal
```

Shadow Deployment (Mirror Traffic)

Shadow deployment sends a copy of every production request to the new version simultaneously. Users only see responses from the current version (shadow responses are discarded). The new version processes real production traffic with zero user impact.

Shadow Testing with Istio

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: payment-service
spec:
  hosts: [payment-service]
  http:
  - route:
    - destination:
        host: payment-service
        subset: v1-stable
      weight: 100
    mirror:
      host: payment-service
      subset: v2-shadow      # Mirror 100% of traffic to v2
    mirrorPercentage:
      value: 100.0

# Users only see v1 responses
# v2 receives identical requests — responses are discarded
# Monitor v2 logs and metrics for errors, latency differences
```

Shadow Testing with NGINX Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: payment-shadow
  annotations:
    nginx.ingress.kubernetes.io/mirror-uri: 'http://payment-v2-svc'
    nginx.ingress.kubernetes.io/mirror-host: payment-v2
spec:
  rules:
  - host: payment.company.com
    http:
      paths:
      - backend:
          service: { name: payment-v1 }
```

Monitoring During Shadow Testing

Compare metrics between production (v1) and shadow (v2) side by side:

```
# Prometheus query — compare error rates
rate(http_requests_total{service='payment-v1',status=~'5..'}[5m])
rate(http_requests_total{service='payment-v2-shadow',status=~'5..'}[5m])

# Compare latency percentiles
histogram_quantile(0.99, rate(http_duration_seconds_bucket{service='payment-v2-shadow'}[5m]))
```

Decision Process

Shadow Test Result	Action
v2 error rate ≈ v1, latency ≈ v1	Proceed with blue-green cutover — high confidence
v2 has higher error rate	Investigate before proceeding — v2 has a bug
v2 has higher latency	Profile and optimise — performance regression
v2 produces different response bodies	Verify if difference is intentional or a bug

> *Tip: Shadow testing is the gold standard for validating critical services but requires careful handling: ensure your database and external APIs can handle 2x the normal request load during shadowing, and that shadow requests are clearly identifiable in audit logs (payment systems have strict audit requirements).*

Q11. A pod is stuck in pending state in your Kubernetes cluster. Walk me through your troubleshooting approach.

Answer

A Pending pod means the Kubernetes scheduler cannot find a suitable node to place it on. The reason is always in the pod's events — systematic inspection reveals the exact cause.

Step 1 — Get the Scheduling Failure Reason

```
kubectl describe pod <pending-pod-name>
# The 'Events' section at the bottom always explains why:
# Warning  FailedScheduling  <time>  default-scheduler
#   0/5 nodes are available: 3 Insufficient cpu,
#   2 node(s) had taint {dedicated:payments} that the pod didn't tolerate
```

Cause 1 — Insufficient Resources (CPU or Memory)

```
# Check node available resources
kubectl describe nodes | grep -A5 'Allocated resources'
# Shows: cpu: 3940m/4000m — nearly all CPU allocated

# Check pod resource requests
kubectl get pod <pod> -o jsonpath='{.spec.containers[*].resources.requests}'

# Solutions:
# 1. Scale up cluster (add nodes)
# 2. Reduce resource requests on the pending pod
# 3. Remove or reschedule lower-priority pods
# 4. Check if cluster autoscaler is working
kubectl logs -n kube-system deployment/cluster-autoscaler | tail -30
```

Cause 2 — Node Taints Not Tolerated

```
# If error mentions 'taint'
kubectl get nodes -o json | jq '.items[] | {name:.metadata.name, taints:.spec.taints}'
```

```
# Add toleration to pod spec
tolerations:
- key: dedicated
  operator: Equal
  value: payments
  effect: NoSchedule
```

Cause 3 — Node Affinity or Pod Anti-Affinity

```
# Pod requires specific node labels that no node has
kubectl get nodes --show-labels
# Check: does any node have topology.kubernetes.io/zone=us-east-1a?

# Pod anti-affinity prevents same-node scheduling
# If all nodes already have one replica, anti-affinity blocks second
```

Cause 4 — PersistentVolumeClaim Not Bound

```
kubectl get pvc
# NAME   STATUS   VOLUME   CAPACITY
# my-pvc Pending  <none>   <none>    # PVC waiting for volume

kubectl describe pvc my-pvc
# Events: no persistent volumes available for this claim
# Fix: check StorageClass exists and provisioner is running
kubectl get storageclass
kubectl get pods -n kube-system | grep provisioner
```

Cause 5 — EBS Volume AZ Mismatch (EKS)

```
# Pod requires EBS volume in us-east-1a
# But only nodes in us-east-1b are available
kubectl describe pod | grep -i 'volume\|topology'
# Fix: use WaitForFirstConsumer binding mode in StorageClass
# or pin pod to correct AZ with nodeAffinity
```

Cause 6 — Image Pull Policy Causing Pending

```
# Pod in Pending with ContainerCreating — different from pure Pending
kubectl get events | grep -i 'pull\|image'
```

Diagnostic Command Summary

```
# One-liner to get all relevant info
kubectl describe pod <pending-pod> | tail -20   # Events section
kubectl get nodes -o wide                       # Node status and IPs
kubectl describe nodes | grep -E 'Taints|Allocatable|cpu:|memory:'
kubectl get events --field-selector reason=FailedScheduling
```

> *Tip: The scheduler's error message in Events is always the answer. Read it carefully — it specifies exactly how many nodes failed and why (insufficient CPU, taint not tolerated, affinity rules). Never guess when the scheduler tells you the exact reason.*

Q12. How would you implement automated upgrades for an EKS cluster with zero downtime?

Answer

EKS cluster upgrades involve four sequential phases: control plane, core add-ons, managed node groups, and Kubernetes workload manifests. Each phase requires verification before proceeding to the next.

Pre-Upgrade Checklist

- Test the upgrade in staging cluster first — always.
- Check Kubernetes version skew policy — max 2 minor versions behind.
- Review the Kubernetes changelog for deprecated APIs: kubectl convert or pluto scan.
- Verify all PodDisruptionBudgets are set on critical workloads.
- Check add-on compatibility: CoreDNS, kube-proxy, VPC CNI, EBS CSI driver.

```
# Check for deprecated APIs in your manifests
pluto detect-helm -r all-namespaces
# Or:
kubectl deprecations
```

Phase 1 — Upgrade Control Plane

```
# Verify current version
aws eks describe-cluster --name prod-cluster \
  --query 'cluster.version'

# Upgrade control plane (AWS manages this — ~20-30 min, no downtime)
aws eks update-cluster-version \
```

```
  --name prod-cluster \
  --kubernetes-version 1.30

# Wait for completion
aws eks wait cluster-active --name prod-cluster
aws eks describe-cluster --name prod-cluster --query 'cluster.status'
# Should show: ACTIVE
```

Phase 2 — Upgrade EKS Add-Ons

```
# List current add-on versions
aws eks list-addons --cluster-name prod-cluster
aws eks describe-addon --cluster-name prod-cluster --addon-name coredns

# Get recommended version for new k8s version
aws eks describe-addon-versions \
  --kubernetes-version 1.30 \
  --addon-name coredns

# Upgrade each add-on
for ADDON in coredns kube-proxy vpc-cni aws-ebs-csi-driver; do
  VERSION=$(aws eks describe-addon-versions \
    --kubernetes-version 1.30 --addon-name $ADDON \
    --query 'addons[0].addonVersions[0].addonVersion' --output text)
  aws eks update-addon --cluster-name prod-cluster \
    --addon-name $ADDON --addon-version $VERSION
  echo "Updated $ADDON to $VERSION"
done
```

Phase 3 — Upgrade Managed Node Groups (Zero Downtime)

AWS performs a rolling node upgrade: launches a new node with the updated AMI, drains one old node, waits for pods to reschedule, then terminates the old node. PodDisruptionBudgets are respected during this process.

```
# Upgrade node group
aws eks update-nodegroup-version \
  --cluster-name prod-cluster \
  --nodegroup-name prod-nodes

# Monitor the upgrade
aws eks describe-nodegroup \
  --cluster-name prod-cluster \
  --nodegroup-name prod-nodes \
  --query 'nodegroup.updateConfig'

# Watch pods during drain
watch kubectl get pods --all-namespaces
```

Ensuring Zero Downtime During Node Upgrade

Workloads must be configured to tolerate node drain. Apply these to all critical deployments before upgrading:

```
# PodDisruptionBudget — ensures minimum availability during drain
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 2      # Always keep at least 2 pods running
  selector:
    matchLabels:
      app: api-server

# Pod spread across AZs — no single node failure takes down service
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels: { app: api-server }
```

Phase 4 — Update kubeconfig and Verify

```
aws eks update-kubeconfig --name prod-cluster --region us-east-1
kubectl version
kubectl get nodes    # All nodes on new version
kubectl get pods --all-namespaces    # All pods running
```

> *Tip: Upgrade one minor version at a time (1.27→1.28→1.29). Skipping versions is not supported. Plan approximately 1 hour per version upgrade including verification. Schedule upgrades during low-traffic windows even if the process itself is zero-downtime.*

Q13. Explain the challenges with StatefulSets and how you'd handle persistent storage in Kubernetes.

Answer

StatefulSets are powerful but complex. Understanding their challenges prevents data loss, split-brain scenarios, and deployment failures in production stateful workloads.

Challenge 1 — PVC Lifecycle is Independent of StatefulSet

When you delete a StatefulSet, its PersistentVolumeClaims are NOT deleted. This is intentional (data protection) but can cause confusion — and old PVCs prevent pods from starting if they conflict with new PVC templates:

```
# PVCs persist after StatefulSet deletion
kubectl delete statefulset postgres
kubectl get pvc    # data-postgres-0, data-postgres-1 still exist!

# When you recreate the StatefulSet, pods reattach to existing PVCs
# This means data is preserved (good) but old data may conflict (bad)

# To fully clean up (DESTROYS DATA):
kubectl delete statefulset postgres
kubectl delete pvc -l app=postgres
```

Challenge 2 — Pods Start and Stop in Order

StatefulSets start pods in order (0, 1, 2) and won't start pod-1 until pod-0 is Running and Ready. If pod-0 has an issue, all subsequent pods are blocked. This serial startup is intentional for clustered databases but can make troubleshooting feel slow:

```
# If postgres-0 is stuck, postgres-1 and postgres-2 won't start
kubectl get pods
# postgres-0   0/1   CrashLoopBackOff
# postgres-1   0/1   Pending            # Waiting for postgres-0

# Fix postgres-0 first, then the others start automatically
```

Challenge 3 — AZ Affinity and EBS Volumes

EBS volumes are AZ-specific. If a StatefulSet pod reschedules to a different AZ (after a node failure), it can't attach its EBS volume. The pod stays Pending:

```
# Fix: Use WaitForFirstConsumer binding mode
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-wait
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer  # Volume created in same AZ as pod
parameters:
  type: gp3
  iops: '3000'
  throughput: '125'
```

Challenge 4 — No Rolling Update Safety for Leader/Replica Topology

For databases with a primary/replica topology, Kubernetes doesn't know which pod is the leader. A rolling update that restarts the primary first can cause a brief outage. Solution: use podManagementPolicy and partition for controlled updates:

```
spec:
  podManagementPolicy: OrderedReady  # Default — safest for databases
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 2   # Only update pods with ordinal >= 2
      # Update replica pods first, verify, then set partition: 0 to update primary
```

Handling Persistent Storage — Best Practices

StorageClass Configuration for Production

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain      # Don't auto-delete volumes on PVC deletion
allowVolumeExpansion: true # Allow online volume expansion
parameters:
  type: gp3
  iops: '6000'
```

```
  encrypted: 'true'
```

StatefulSet VolumeClaimTemplate

```
volumeClaimTemplates:
- metadata:
    name: data
  spec:
    accessModes: [ReadWriteOnce]
    storageClassName: fast-ssd
    resources:
      requests:
        storage: 100Gi
```

Backup Strategy for StatefulSet Data

- Velero — Kubernetes-native backup solution that snapshots PVs and backs up all Kubernetes resources:

```
velero backup create postgres-backup \
  --include-namespaces database \
  --snapshot-volumes
```

- Database-native backups — For PostgreSQL, pg_dump via CronJob; for MySQL, mysqldump. Store in S3.
- CSI Volume Snapshots — Take point-in-time snapshots at the storage level:

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: postgres-snapshot
spec:
  volumeSnapshotClassName: csi-aws-vsc
  source:
    persistentVolumeClaimName: data-postgres-0
```

Tip: For production databases, seriously consider managed cloud services (RDS, Cloud SQL, Aurora) over StatefulSets. The operational complexity of managing database HA, backups, failover, and upgrades in Kubernetes is significant. StatefulSets are best for workloads where managed services don't exist or are too expensive.

Q14. Design a comprehensive monitoring stack for Kubernetes workloads. What tools would you use and why?

Answer

A production Kubernetes monitoring stack addresses three pillars: metrics (is something wrong?), logs (what happened?), and traces (where is the bottleneck?). Each pillar requires specific tools.

Complete Monitoring Stack Architecture

```
Metrics:  Prometheus → Grafana → PagerDuty
Logs:     Fluent Bit (DaemonSet) → Loki or CloudWatch Logs → Grafana
Traces:   OpenTelemetry SDK → Tempo or Jaeger → Grafana
Alerts:   Alertmanager → Slack, PagerDuty, email
Synthetic: Blackbox Exporter → Grafana → PagerDuty
```

1. Metrics — Prometheus + Grafana

Prometheus scrapes metrics from pods (via /metrics endpoints), stores them as time-series, and evaluates alert rules. Grafana visualises them.

```
# Install kube-prometheus-stack (includes everything)
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts

helm install kube-prometheus-stack \
  prometheus-community/kube-prometheus-stack \
  -n monitoring \
  -f values.yaml

# values.yaml key settings
prometheus:
  prometheusSpec:
    retention: 30d
    storageSpec:
      volumeClaimTemplate:
        spec:
          resources:
            requests:
              storage: 100Gi
    additionalScrapeConfigs:
    - job_name: 'app-services'
```

```
      kubernetes_sd_configs:
      - role: pod
      relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
```

2. Auto-Discovery with ServiceMonitor

Applications expose Prometheus metrics — define a ServiceMonitor to tell Prometheus to scrape them:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-monitor
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
  - port: metrics
    path: /metrics
    interval: 15s
```

3. Alerting — PrometheusRule

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: critical-alerts
spec:
  groups:
  - name: pod-health
    rules:
    - alert: PodCrashLooping
      expr: |
        rate(kube_pod_container_status_restarts_total[5m]) > 0
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: 'Pod {{ $labels.pod }} is crash looping'

    - alert: HighErrorRate
      expr: |
        rate(http_requests_total{status=~'5..'}[5m])
          / rate(http_requests_total[5m]) > 0.01
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: 'Error rate above 1% for {{ $labels.service }}'

    - alert: NodeMemoryPressure
      expr: kube_node_status_condition{condition='MemoryPressure',status='true'} == 1
      for: 1m
      labels:
        severity: page
```

4. Logging — Fluent Bit + Loki

```
# Fluent Bit as DaemonSet — collects logs from every node
helm install fluent-bit fluent/fluent-bit \
  -n monitoring \
  --set config.outputs[0].Match=*
  --set config.outputs[0].host=loki
  --set config.outputs[0].port=3100

# In Grafana: query logs with LogQL
# {namespace='production', app='payment-service'} |= 'ERROR'
```

5. Distributed Tracing — OpenTelemetry + Tempo

```
# Application instrumentation (Python example)
from opentelemetry import trace
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace import TracerProvider

tracer = trace.get_tracer('payment-service')
```

```
with tracer.start_as_current_span('process-payment') as span:
    span.set_attribute('payment.amount', amount)
    span.set_attribute('payment.currency', currency)
    result = process_payment(payment_data)
```

6. Grafana Dashboards

Key dashboards to create or import from grafana.com:

- Kubernetes cluster overview (dashboard ID 7249)
- Node resource usage (dashboard ID 1860)
- Pod resource requests/limits compliance
- Application RED metrics (Rate, Errors, Duration) per service
- SLO burn rate dashboard for error budget tracking

Tool	Purpose
Prometheus	Metrics collection and alerting
Grafana	Visualisation and dashboards
Alertmanager	Alert routing (Slack, PagerDuty)
Fluent Bit	Log collection (DaemonSet)
Loki	Log aggregation and search
Tempo/Jaeger	Distributed tracing
OpenTelemetry	Instrumentation standard

> *Tip: Start with kube-prometheus-stack — one Helm chart installs Prometheus, Grafana, Alertmanager, node-exporter, and kube-state-metrics with pre-built Kubernetes dashboards and alerts. It's the fastest path to comprehensive Kubernetes monitoring.*

Q15. How would you handle complex scaling scenarios for Kubernetes workloads with varying traffic patterns?

Answer

Real-world traffic is never flat. E-commerce spikes at noon, gaming peaks at evening, CI/CD jobs run in bursts. Complex scaling combines multiple Kubernetes autoscaling mechanisms to handle these patterns efficiently.

The Three Scaling Dimensions

Dimension	Mechanism
Horizontal Pod Autoscaler (HPA)	Adjusts replica count
Vertical Pod Autoscaler (VPA)	Adjusts resource requests
Cluster Autoscaler / Karpenter	Adds/removes nodes

Scenario 1 — Predictable Daily Traffic (Business Hours)

Traffic follows a daily pattern. Use scheduled scaling (KEDA CronScaler) to pre-scale before the peak:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: api-scheduled-scaler
spec:
  scaleTargetRef:
    name: api-deployment
  triggers:
  - type: cron
    metadata:
      timezone: Asia/Kolkata
      start: '0 8 * * 1-5'    # Scale up at 8 AM weekdays
      end:   '0 21 * * 1-5'   # Scale down at 9 PM
      desiredReplicas: '20'    # Pre-scale to 20 pods at 8 AM
```

Scenario 2 — Burst Traffic (Queue-Driven)

Scale based on queue depth — use KEDA to scale on SQS queue length, Kafka topic lag, or Redis list length:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: worker-sqs-scaler
spec:
  scaleTargetRef:
    name: message-processor
  minReplicaCount: 0     # Scale to zero when queue is empty
  maxReplicaCount: 100
  triggers:
  - type: aws-sqs-queue
    metadata:
      queueURL: https://sqs.us-east-1.amazonaws.com/123/my-queue
      queueLength: '10'  # 1 pod per 10 messages in queue
      awsRegion: us-east-1
```

Scenario 3 — Multi-Metric Scaling

Scale on multiple metrics simultaneously — take the highest replica count:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: payment-service
  minReplicas: 5
  maxReplicas: 100
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  - type: External
    external:
      metric:
        name: sqs_messages_visible
        selector:
          matchLabels:
            queue: payment-queue
      target:
        type: AverageValue
        averageValue: '5'
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: '100'
```

Scenario 4 — Spike Traffic with Fast Scale-Up

Adjust HPA behavior for aggressive scale-up, conservative scale-down:

```
spec:
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0   # Scale up immediately
      policies:
      - type: Percent
        value: 200       # Can double replica count per period
        periodSeconds: 15
      - type: Pods
        value: 10        # Or add 10 pods per period, whichever is more
        periodSeconds: 15
      selectPolicy: Max  # Use the policy that gives more pods
    scaleDown:
      stabilizationWindowSeconds: 600  # Wait 10 min before scaling down
      policies:
      - type: Percent
        value: 10        # Remove max 10% per period
        periodSeconds: 60
```

Scenario 5 — Node-Level Scaling with Karpenter

```
apiVersion: karpenter.sh/v1alpha5
```

```
kind: Provisioner
metadata:
  name: default
spec:
  requirements:
  - key: karpenter.sh/capacity-type
    operator: In
    values: [spot, on-demand]
  - key: node.kubernetes.io/instance-type
    operator: In
    values: [m5.large, m5.xlarge, m5.2xlarge, m4.xlarge]
  limits:
    resources:
      cpu: '1000'        # Max 1000 vCPUs in cluster
  ttlSecondsAfterEmpty: 30  # Remove empty nodes after 30s
```

Tip: Combine KEDA (for event-driven and scheduled scaling) with HPA (for metric-based scaling) and Karpenter (for node scaling). KEDA and HPA can coexist on the same deployment — KEDA manages the ScaledObject which creates an HPA underneath.

Q16. How do you implement GitOps in a Kubernetes environment?

Answer

GitOps is an operational model where Git is the single source of truth for both application and infrastructure state. Changes are made via Git commits (not kubectl apply), and a controller continuously reconciles the cluster state with the Git repository.

Core GitOps Principles

- Declarative — All desired system state is expressed declaratively in Git.
- Versioned and immutable — Git provides history, auditability, and rollback.
- Pulled automatically — A controller (Argo CD, Flux) pulls desired state from Git and applies it — no push-based kubectl from CI/CD.
- Continuously reconciled — The controller constantly compares actual vs desired state and corrects drift.

Tool 1 — Argo CD

Argo CD is a declarative GitOps continuous delivery tool for Kubernetes. It monitors a Git repository and automatically syncs applications:

```
# Install Argo CD
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Define an Application — tells Argo CD what to watch
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: payment-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/company/k8s-manifests
    targetRevision: main
    path: apps/payment-service/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true         # Delete resources removed from Git
      selfHeal: true      # Auto-fix drift (kubectl manual changes)
    syncOptions:
    - CreateNamespace=true
    retry:
      limit: 3
      backoff: { duration: 5s, maxDuration: 3m }
```

Git Repository Structure for GitOps

```
k8s-manifests/
├── apps/
```

```
│   ├── payment-service/
│   │   ├── base/              # Kustomize base
│   │   │   ├── deployment.yaml
│   │   │   ├── service.yaml
│   │   │   └── kustomization.yaml
│   │   ├── dev/
│   │   │   └── kustomization.yaml  # Patches for dev
│   │   └── production/
│   │       └── kustomization.yaml  # Patches for prod
│   └── api-gateway/
├── infrastructure/
│   ├── cert-manager/
│   ├── ingress-nginx/
│   └── monitoring/
└── clusters/
    ├── dev-cluster/
    │   └── apps.yaml       # Argo CD ApplicationSet
    └── prod-cluster/
        └── apps.yaml
```

CI/CD Pipeline Flow with GitOps

```
Developer pushes code → GitHub Actions builds image → tags with git SHA
→ Updates image tag in k8s-manifests repo (automated PR or direct commit)
→ Argo CD detects change in Git → syncs to cluster
→ Kubernetes rolls out new pods

# CI pipeline step — update image tag in GitOps repo
- name: Update GitOps repo
  run: |
    git clone https://github.com/company/k8s-manifests
    cd k8s-manifests
    # Update image tag using kustomize
    cd apps/payment-service/production
    kustomize edit set image payment=myacr.io/payment:${{ github.sha }}
    git commit -am 'Update payment-service to ${{ github.sha }}'
    git push
    # Argo CD picks up the change and deploys automatically
```

Rollback — The GitOps Way

Rollback in GitOps is a Git revert — not kubectl rollout undo. This creates a new commit that restores the previous state, preserving audit history:

```
# Rollback by reverting the Git commit
git revert HEAD --no-edit
git push
# Argo CD detects the new commit and deploys the reverted state

# Or: Argo CD UI → Application → History → Rollback to revision N
```

> *Tip: Use Argo CD ApplicationSets to manage many applications across many clusters from a single template. One ApplicationSet can deploy 50 microservices to 5 clusters with environment-specific overrides — far more scalable than managing individual Application resources.*

Q17. Can you explain how you would create a fully automated blue-green deployment in a Kubernetes-based microservices architecture?

Answer

A fully automated blue-green deployment in a microservices architecture automates the entire lifecycle: build, deploy green, test, switch, verify, and clean up — without manual intervention. See Q4 for the basic mechanism. This answer focuses on full automation across multiple services.

Architecture for Multi-Service Blue-Green

In a microservices architecture, blue-green is applied per-service independently. Each service has its own blue and green deployment, managed by a CI/CD pipeline that coordinates between them.

```
Service A (auth):     blue=v1.2, green=v1.3 (testing)
Service B (payments): blue=v3.1, green=v3.1 (no change)
Service C (orders):   blue=v2.5, green=v2.6 (deploying)
```

Automated Pipeline — GitHub Actions

```
name: Blue-Green Deploy
on:
  push:
```

```
    branches: [main]

jobs:
  blue-green-deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Determine active slot
      id: slot
      run: |
        CURRENT=$(kubectl get service my-service \
          -o jsonpath='{.spec.selector.version}')
        if [ "$CURRENT" = 'blue' ]; then
          echo 'deploy_slot=green' >> $GITHUB_OUTPUT
          echo 'cutover_slot=blue' >> $GITHUB_OUTPUT
        else
          echo 'deploy_slot=blue' >> $GITHUB_OUTPUT
          echo 'cutover_slot=green' >> $GITHUB_OUTPUT
        fi

    - name: Deploy to inactive slot
      run: |
        SLOT=${{ steps.slot.outputs.deploy_slot }}
        kubectl set image deployment/my-service-$SLOT \
          app=myapp:${{ github.sha }}
        kubectl rollout status deployment/my-service-$SLOT \
          --timeout=300s

    - name: Run automated smoke tests against new slot
      run: |
        SLOT=${{ steps.slot.outputs.deploy_slot }}
        POD=$(kubectl get pod -l version=$SLOT \
          -o jsonpath='{.items[0].metadata.name}')
        # Test endpoints directly on the new slot
        kubectl exec $POD -- curl -sf localhost:8080/health
        kubectl exec $POD -- python3 /tests/smoke_test.py

    - name: Switch traffic to new slot
      run: |
        SLOT=${{ steps.slot.outputs.deploy_slot }}
        kubectl patch service my-service \
          -p "{\"spec\":{\"selector\":{\"version\":\"$SLOT\"}}}"

    - name: Monitor for 5 minutes
      run: |
        sleep 300
        # Check error rate via Prometheus
        ERROR_RATE=$(curl -s 'http://prometheus/api/v1/query?query=
          rate(http_requests_total{status=~"5.."}[5m])/rate(http_requests_total[5m])'
          | jq '.data.result[0].value[1]')
        if (( $(echo "$ERROR_RATE > 0.01" | bc -l) )); then
          echo 'ERROR RATE TOO HIGH — rolling back'
          PREV=${{ steps.slot.outputs.cutover_slot }}
          kubectl patch service my-service \
            -p "{\"spec\":{\"selector\":{\"version\":\"$PREV\"}}}"
          exit 1
        fi
        echo 'Deployment successful — cleaning up old slot'

    - name: Scale down old slot
      if: success()
      run: |
        OLD=${{ steps.slot.outputs.cutover_slot }}
        kubectl scale deployment my-service-$OLD --replicas=1
        # Keep 1 replica of old slot for fast rollback
```

Using Argo Rollouts for Fully Managed Blue-Green

Argo Rollouts automates the entire blue-green process declaratively:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: payment-service
spec:
  replicas: 5
  strategy:
    blueGreen:
```

```
    activeService: payment-active
    previewService: payment-preview
    autoPromotionEnabled: false  # Manual approval before switch
    prePromotionAnalysis:
      templates:
      - templateName: smoke-tests
    postPromotionAnalysis:
      templates:
      - templateName: error-rate-check
    scaleDownDelaySeconds: 600   # Keep blue for 10 min after switch
```

Tip: Use Argo Rollouts for microservices blue-green in production — it handles the complexity of traffic switching, analysis, rollback, and cleanup automatically, and integrates with Prometheus and Datadog for data-driven promotion decisions.

Q18. How do you manage secrets and config securely at scale in Kubernetes without compromising GitOps workflows?

Answer

This is covered comprehensively in the Azure & DevOps answers (Q14 of that document) and overlaps with Terraform Q36. The Kubernetes-specific answer focuses on the available tools and their GitOps compatibility.

The Core Problem

GitOps requires everything in Git. But Kubernetes Secrets are only base64-encoded — not encrypted. Committing them to Git exposes credentials to anyone with repository access. The solution: store encrypted representations in Git, with actual secret values in an external store.

Solution 1 — Sealed Secrets (Bitnami)

Sealed Secrets encrypt Kubernetes Secret objects using the cluster's public key. Only the cluster can decrypt. Safe to commit to Git.

```
# Install kubeseal CLI and controller
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system

# Seal a secret
kubectl create secret generic db-creds \
  --from-literal=password='s3cr3t' \
  --dry-run=client -o yaml | \
  kubeseal --controller-namespace kube-system \
           --format yaml > db-creds-sealed.yaml

# Commit db-creds-sealed.yaml to Git — safe!
# Argo CD syncs it; cluster controller decrypts and creates K8s Secret
```

Solution 2 — External Secrets Operator (ESO)

ESO syncs values from AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault into Kubernetes Secrets. The ExternalSecret definition (not the values) lives in Git:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: payment-db-creds
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: payment-db-creds
    creationPolicy: Owner
  data:
  - secretKey: db-password
    remoteRef:
      key: prod/payment/db-credentials
      property: password
```

Solution 3 — Secrets Store CSI Driver

Mounts secrets directly as files from an external store into pods — no Kubernetes Secret object created:

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: payment-secrets
spec:
```

```
  provider: aws
  parameters:
    objects: |
      - objectName: 'prod/payment/db-credentials'
        objectType: secretsmanager
        objectAlias: db-password
```

Solution 4 — SOPS (Secrets OPerationS)

SOPS encrypts secret values in YAML files using AWS KMS or age keys. Commit the encrypted file — CI/CD and Argo CD (with SOPS plugin) decrypt at deploy time:

```
# Encrypt
sops --kms arn:aws:kms:us-east-1:123:key/xxx --encrypt secret.yaml > secret.enc.yaml

# Argo CD + SOPS plugin decrypts during sync
```

Solution	GitOps Compatible
Sealed Secrets	Yes — safe to commit
External Secrets Operator	Yes — ExternalSecret in Git
Secrets Store CSI	Yes — SecretProviderClass in Git
SOPS + KMS	Yes — encrypted YAML in Git

Tip: For new GitOps setups, External Secrets Operator with AWS Secrets Manager is the most production-ready choice. It handles automatic secret rotation, multi-cluster sync, and provides a clean separation — your DevOps team manages secrets in Secrets Manager, and GitOps manages the ExternalSecret manifests.

Q19. Explain the control plane components of Kubernetes and how you would harden them for production use.

Answer

The Kubernetes control plane is the brain of the cluster. Understanding each component's role and securing it is essential for production-grade clusters.

Control Plane Components

Component	Role
kube-apiserver	Central communication hub — all kubectl, kubelet, controllers interact through it
etcd	Distributed key-value store — stores ALL cluster state
kube-scheduler	Assigns pods to nodes based on resource availability and constraints
kube-controller-manager	Runs control loops: ReplicaSet, Deployment, Node controllers
cloud-controller-manager	Cloud-specific controllers: LoadBalancer, PersistentVolume provisioning

kube-apiserver — The API Gateway

Every interaction with the cluster goes through the API server. It authenticates, authorises, and validates all requests. Hardening:

- Enable audit logging — Record every API call for compliance:

```
--audit-log-path=/var/log/kubernetes/audit.log
--audit-log-maxage=30
--audit-policy-file=/etc/kubernetes/audit-policy.yaml
```

- Disable anonymous access: --anonymous-auth=false
- Enable admission controllers:

```
--enable-admission-plugins=NodeRestriction,PodSecurity,ResourceQuota,\
LimitRanger,ServiceAccount
```

- Restrict API server access — only allow access from known CIDR ranges:

```
--authorization-mode=Node,RBAC
```

etcd — The Crown Jewel

etcd stores all cluster state including Secrets. If etcd is compromised, the entire cluster is compromised. Hardening:

- Encrypt etcd at rest — Kubernetes Secrets are stored in etcd in plaintext without this:

```
# /etc/kubernetes/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources: [secrets, configmaps]
  providers:
  - aescbc:
      keys:
      - name: key1
        secret: <base64-encoded-32-byte-key>
  - identity: {}   # Fallback for reading unencrypted data
```

- mTLS between etcd and API server — Only the API server should communicate with etcd.
- etcd backup — Automated snapshots every 5-30 minutes:

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%Y%m%d%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

RBAC — Cluster-Wide Access Control

```
# Least privilege for service accounts
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: ['']
  resources: [pods]
  verbs: [get, list, watch]
  # No create, delete, update — read only
```

Pod Security Standards

```
# Apply restricted policy to production namespace
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# Restricted profile requires:
# - runAsNonRoot: true
# - allowPrivilegeEscalation: false
# - readOnlyRootFilesystem: true
# - No host namespace access
# - Seccomp profile: RuntimeDefault or Localhost
```

CIS Benchmark — Automated Compliance

```
# Run kube-bench to check against CIS Kubernetes Benchmark
kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
kubectl logs job/kube-bench
# Identifies misconfigurations against CIS controls
```

> *Tip: For managed Kubernetes (EKS, AKS, GKE), the cloud provider manages control plane component security. Focus on: enabling audit logging, etcd encryption at rest, RBAC least privilege, Pod Security Standards, and network policies. These four controls address the majority of Kubernetes security risks.*

Q20. How would you scale a Kubernetes cluster horizontally across multiple regions and still ensure zero-downtime upgrades?

Answer

Multi-region Kubernetes requires solving two distinct problems: routing traffic globally with failover, and managing coordinated upgrades across regions without downtime.

Multi-Region Architecture

```
Users → Global Load Balancer (Route 53 / Azure Traffic Manager)
  ↓                              ↓
Region: us-east-1           Region: eu-west-1
```

```
EKS Cluster (primary)       EKS Cluster (secondary)
  ↓                              ↓
App Pods + Services         App Pods + Services
  ↓                              ↓
RDS Multi-AZ               RDS Multi-AZ (read replica from primary)
```

Global Traffic Routing — Route 53

```
# Latency-based routing — users go to nearest healthy region
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123 \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "api.company.com",
        "Type": "A",
        "SetIdentifier": "us-east-1",
        "Region": "us-east-1",
        "AliasTarget": {
          "DNSName": "<us-east-1-alb-dns>",
          "EvaluateTargetHealth": true
        }
      }
    }]
  }'
```

Cluster Federation — Argo CD ApplicationSet

Use Argo CD to deploy the same application to multiple clusters from one definition:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: payment-service-global
spec:
  generators:
  - clusters:
      selector:
        matchLabels:
          environment: production
  template:
    metadata:
      name: '{{name}}-payment-service'
    spec:
      project: default
      source:
        repoURL: https://github.com/company/k8s-manifests
        path: 'apps/payment-service/{{metadata.labels.region}}'
      destination:
        server: '{{server}}'
        namespace: production
```

Zero-Downtime Multi-Region Upgrade Strategy

Upgrade regions sequentially to maintain service availability:

```
Phase 1: Upgrade secondary region (eu-west-1)
  - Route 10% traffic to secondary via weighted DNS
  - Run full upgrade on secondary cluster
  - Verify health for 30 minutes
  - If issues: route all traffic back to primary (instant)

Phase 2: Traffic shift
  - Route 50% to each region
  - Monitor error rates in both regions

Phase 3: Upgrade primary (us-east-1)
  - Route 90% traffic to secondary temporarily
  - Upgrade primary cluster
  - Verify health

Phase 4: Restore balanced routing
  - 50/50 split across both upgraded regions
```

Cross-Region Considerations

- Database replication — Use RDS Global Database or DynamoDB Global Tables for sub-second cross-region replication. Active-active vs active-passive depends on write consistency requirements.
- Session affinity — If stateful sessions, use sticky routing or move sessions to Redis (ElastiCache Global Datastore).

- Config consistency — Use Parameter Store or Secrets Manager cross-region replication. The same secret name resolves to the regional secret.
- Image registry — Use ECR replication rules to mirror images to each region before deployment.

> *Tip: Start with a warm standby multi-region setup (primary active, secondary passive with same infrastructure) before attempting active-active. Active-active with distributed databases requires careful consistency management and significantly increases complexity.*

Q21. What is a PodDisruptionBudget and how do you use it in critical workloads?

Answer

A PodDisruptionBudget (PDB) limits the number of pods of an application that can be voluntarily disrupted simultaneously. It ensures that a minimum number of pods remain available during planned disruptions — node drains, cluster upgrades, or rolling updates.

What is a Voluntary Disruption

Voluntary disruptions are planned actions that terminate pods: node drain (kubectl drain), cluster upgrades (EKS node group update), manual kubectl delete pod, or Cluster Autoscaler removing an underutilised node. PDBs protect against these. PDBs do NOT protect against involuntary disruptions (node hardware failure, kernel panic).

PDB Configuration

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: payment-service-pdb
  namespace: production
spec:
  # Option A: minAvailable — minimum pods that MUST be running
  minAvailable: 3

  # Option B: maxUnavailable — max pods that CAN be unavailable
  # maxUnavailable: 1

  selector:
    matchLabels:
      app: payment-service
```

minAvailable vs maxUnavailable

Setting	Example
minAvailable: 3	At least 3 must be running
minAvailable: '80%'	80% must be running
maxUnavailable: 1	Only 1 can be unavailable at a time
maxUnavailable: '20%'	20% can be unavailable

Real-World Usage — Payment Service

```
# Payment service: 5 replicas — must never go below 4
spec:
  minAvailable: 4
  selector:
    matchLabels:
      app: payment-service

# Database StatefulSet: 3 nodes — always keep majority
spec:
  minAvailable: 2    # Quorum: 2 of 3 always up
  selector:
    matchLabels:
      app: postgres
```

How PDB Works During Node Drain

```
# Node drain with PDB enforcement
kubectl drain node-3 --ignore-daemonsets --delete-emptydir-data

# If payment-service PDB says minAvailable=4 and 4 pods are on other nodes:
# Node drain can terminate payment-service pod on node-3 (4 remain)
```

```
# If only 4 pods total and 2 are on node-3:
# Node drain PAUSES — cannot terminate without violating PDB
# evicting pod/payment-service-abc123: error when evicting pods
# Cannot evict pod as it would violate the pod's disruption budget.
```

Checking PDB Status

```
kubectl get pdb -n production
# NAME                 MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS  AGE
# payment-service-pdb   4              N/A              1                    5d
# ALLOWED DISRUPTIONS=1 means: currently safe to disrupt 1 pod

kubectl describe pdb payment-service-pdb
# Shows: current pod count, disruptions allowed, status conditions
```

Common Mistakes

- Setting minAvailable equal to replica count — No disruptions are ever allowed. Node drains hang indefinitely:

```
replicas: 3
minAvailable: 3   # WRONG — no pod can ever be disrupted
                  # Node drain will block forever
```

- Not setting PDB on StatefulSets — Database node drains can terminate multiple replicas simultaneously, causing quorum loss.
- PDB with minAvailable: 1 on single-replica deployments — Gives false safety. 1 pod must remain, but if that pod is on the draining node, the drain still blocks.

> *Tip: Set PDBs on all production deployments with 2+ replicas. For single-replica services, scale to at least 2 replicas and set minAvailable: 1. A single-replica service with no PDB can be completely disrupted by a node drain.*

Q22. How do you implement and manage network policies in Kubernetes for strict inter-service communication?

Answer

This expands on Q5 with a focus on enterprise-scale implementation, namespace isolation strategies, and operationalising network policies across many services.

Namespace-Level Isolation Strategy

Isolate namespaces so that traffic between namespaces is denied by default and must be explicitly allowed. This is the foundation of zero-trust networking in Kubernetes:

```
# Apply to every namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}    # All pods
  policyTypes: [Ingress, Egress]
  # No rules = deny everything

# Then selectively allow intra-namespace communication
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-same-namespace
spec:
  podSelector: {}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: {}   # Allow all pods within this namespace
```

Service-to-Service Communication Matrix

Define a communication matrix for your microservices, then implement as NetworkPolicies:

Source Service	Destination Service
api-gateway	auth-service
api-gateway	payment-service
payment-service	postgres
payment-service	redis

auth-service	postgres
prometheus	all services
any service	api-gateway

Implementing the Communication Matrix

```
# payment-service can reach postgres only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: payment-to-postgres
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Egress]
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: postgres
    ports:
    - port: 5432
  # Always allow DNS
  - to:
    - namespaceSelector: {}
    ports:
    - port: 53
      protocol: UDP
```

Policy Testing with Netshoot

```
# Deploy test pod for network testing
kubectl run test --image=nicolaka/netshoot --rm -it -- sh

# From test pod — test connectivity
nc -zv payment-service.production 8080
# Connection refused → blocked by NetworkPolicy (expected)

# From api-gateway pod — should succeed
kubectl exec -it api-gateway-pod -- nc -zv payment-service 8080
```

Audit and Visualisation

```
# Use Cilium Hubble for network flow visibility
hubble observe --namespace production --type drop
# Shows: flows that were DROPPED by network policy
# Helps identify missing allow rules vs legitimate blocks
```

> *Tip: Use a GitOps repository to manage all NetworkPolicy resources. This ensures policies are version-controlled, code-reviewed, and automatically applied. Never manage NetworkPolicies manually via kubectl — they're security controls and must be auditable.*

Q23. How would you integrate runtime threat detection in Kubernetes using tools like Falco or Sysdig?

Answer

Runtime threat detection monitors what's happening inside running containers and on Kubernetes nodes in real-time — detecting attacks, policy violations, and suspicious behaviour that static scanning cannot catch.

Why Runtime Detection is Needed

Static image scanning catches known vulnerabilities at build time. But it cannot detect: an attacker who gained access to a running container, a pod that is crypto-mining, a process that is reading Kubernetes secrets, or an unexpected outbound connection to a C2 server. Runtime detection catches these behaviours as they happen.

Falco — Open Source Runtime Security

Falco is a Cloud Native Computing Foundation (CNCF) project that uses eBPF to monitor system calls and Kubernetes API events. It detects suspicious activity based on rules and sends alerts.

```
# Install Falco via Helm
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco \
  -n falco \
```

```
  --set driver.kind=ebpf \
  --set falcosidekick.enabled=true \
  --set falcosidekick.config.slack.webhookurl=$SLACK_WEBHOOK
```

Example Falco Rules

```
# Custom rules file — /etc/falco/rules.d/custom-rules.yaml

# Alert: Shell spawned inside a container
- rule: Terminal shell in container
  desc: A shell was spawned in a container
  condition: >
    spawned_process and container
    and shell_procs
    and not user_expected_terminal_shell_in_container_conditions
  output: >
    Shell spawned in container (pod=%k8s.pod.name ns=%k8s.ns.name
    image=%container.image.repository user=%user.name cmd=%proc.cmdline)
  priority: WARNING
  tags: [container, shell, T1059]

# Alert: Reading sensitive files
- rule: Read sensitive file
  desc: An attempt to read a sensitive file
  condition: >
    open_read and container
    and (fd.name startswith /etc/shadow
      or fd.name startswith /etc/kubernetes/admin.conf),
  output: >
    Sensitive file opened (file=%fd.name pod=%k8s.pod.name)
  priority: CRITICAL

# Alert: Unexpected network connection
- rule: Unexpected outbound connection
  desc: A process in a container made an unexpected outbound connection
  condition: >
    outbound and container
    and not allowed_known_ports
    and not proc.name in (allowed_processes)
  output: >
    Unexpected outbound connection (proc=%proc.name port=%fd.rport
    pod=%k8s.pod.name ip=%fd.rip)
  priority: CRITICAL
```

Alert Routing with Falcosidekick

```
# Falcosidekick routes alerts to multiple destinations
falcosidekick:
  config:
    slack:
      webhookurl: $SLACK_WEBHOOK
      minimumpriority: warning
    pagerduty:
      routingKey: $PAGERDUTY_KEY
      minimumpriority: critical
    elasticsearch:
      hostport: http://elasticsearch:9200
      index: falco-events
```

Sysdig Secure — Enterprise Alternative

Sysdig Secure provides managed runtime security with pre-built rule libraries, compliance frameworks (CIS, NIST, PCI-DSS), drift detection, and forensic capture:

- Pre-built rules for MITRE ATT&CK Kubernetes techniques.
- Drift detection — detects when files are modified inside a running container (image drift).
- Container forensics — captures full system call history for incident investigation.
- Network segmentation — maps actual traffic patterns and suggests NetworkPolicies.

Integration with Kubernetes Admission Control

Combine Falco with OPA/Gatekeeper for preventive controls (blocks bad deployments before they start) + Falco for detective controls (alerts on suspicious runtime behaviour):

- OPA/Gatekeeper — Prevents privileged containers, hostNetwork access, and missing security contexts at deploy time.
- Falco — Detects if a running container escalates privileges, spawns unexpected processes, or reads sensitive files.

> *Tip: Start with Falco's built-in rules (they cover 90% of common attack patterns) before writing custom rules. The default ruleset includes: privilege escalation, shell spawning in containers, sensitive file reads, cryptocurrency mining, and Kubernetes API server access from containers.*

Q24. How do you create and manage Kubernetes clusters (using tools like Terraform), and what are the master and worker nodes?

Answer

Kubernetes clusters consist of two node types with distinct responsibilities. Terraform is the standard IaC tool for provisioning managed Kubernetes clusters (EKS, AKS, GKE).

Master Nodes (Control Plane)

Master nodes run the control plane components that manage the cluster. In managed Kubernetes (EKS, AKS), AWS/Azure manages master nodes — you never directly access them. Components running on master nodes: kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager (see Q19 for details).

Worker Nodes (Data Plane)

Worker nodes run the actual application workloads. They run: kubelet (communicates with API server, manages pods on the node), kube-proxy (maintains network rules, handles Service routing), and a container runtime (containerd, CRI-O).

```
# See nodes and their roles
kubectl get nodes
# NAME          STATUS  ROLES          AGE
# master-1      Ready   control-plane  10d
# worker-1      Ready   <none>         10d
# worker-2      Ready   <none>         10d
```

Provisioning EKS with Terraform

```
# main.tf — EKS Cluster
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  # Control plane access
  cluster_endpoint_private_access = true
  cluster_endpoint_public_access  = true
  cluster_endpoint_public_access_cidrs = ["10.0.0.0/8"]

  # Enable IRSA
  enable_irsa = true

  # Managed node groups
  eks_managed_node_groups = {
    general = {
      instance_types = ["m5.xlarge"]
      min_size       = 3
      max_size       = 20
      desired_size   = 5
      disk_size      = 50
    }
    spot = {
      instance_types    = ["m5.large", "m5.xlarge", "m4.xlarge"]
      capacity_type     = "SPOT"
      min_size          = 0
      max_size          = 50
      desired_size      = 0
    }
  }

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}
```

Post-Provisioning Configuration

```
# Update kubeconfig
aws eks update-kubeconfig \
  --name prod-eks \
```

```
  --region us-east-1

# Verify cluster
kubectl cluster-info
kubectl get nodes -o wide

# Install essential add-ons
helm install aws-load-balancer-controller \
  eks/aws-load-balancer-controller -n kube-system
helm install cluster-autoscaler autoscaler/cluster-autoscaler -n kube-system
helm install metrics-server metrics-server/metrics-server -n kube-system
```

Tip: Use the official terraform-aws-modules/eks/aws module for EKS. It handles the complex IAM role and OIDC configuration, security groups, and node group setup that would take hundreds of lines of raw Terraform resources.

Q25. What are common Kubernetes errors you've faced (like CrashLoopBackOff, ImagePullError), and how did you resolve them?

Answer

Knowing how to diagnose and resolve common Kubernetes errors quickly is a core DevOps competency. Each error has a distinct root cause and targeted fix.

1. CrashLoopBackOff

The container starts, crashes, and restarts repeatedly. Kubernetes waits increasingly longer between restarts (back-off).

```
kubectl describe pod <pod> | grep -A5 'Last State'
kubectl logs <pod> --previous   # Logs from BEFORE the crash
```

Root Cause	Diagnosis
Application error	kubectl logs --previous shows exception
Missing env var/config	Logs show 'config not found' or null pointer
OOMKilled	Last State shows reason: OOMKilled
Wrong command/entrypoint	kubectl describe shows last exit code
Readiness probe killing it	kubectl describe shows readiness probe failed

2. ImagePullBackOff / ErrImagePull

```
kubectl describe pod <pod> | grep -A10 Events
# Failed to pull image: 404 Not Found
# Failed to pull image: 401 Unauthorized
```

Error	Fix
Image tag doesn't exist	kubectl set image deploy/app app=image:correct-tag
Private registry — no credentials	Create imagePullSecret and add to pod spec
Registry unreachable	Check network/VPC connectivity to registry endpoint
ECR auth expired	Recreate ECR token: aws ecr get-login-password | docker login...

3. Pending

Pod cannot be scheduled — covered in detail in Q11. Quick summary:

```
kubectl describe pod | grep -A5 Events
# 0/3 nodes: Insufficient cpu, 1 node: taint...
# Fix based on error: add resources, add toleration, adjust affinity
```

4. OOMKilled

```
kubectl describe pod | grep 'OOMKilled'
kubectl top pods   # Check actual memory usage
# Fix: increase memory limit or fix memory leak
kubectl set resources deployment my-app --limits=memory=512Mi
```

5. Evicted

```
kubectl describe pod | grep -i evict
kubectl describe node | grep -i pressure
# Fix: clean up node disk, add resource limits, scale nodes
```

6. CreateContainerConfigError

```
# ConfigMap or Secret referenced in pod spec doesn't exist
kubectl describe pod | grep 'Error'
# Error: secret 'db-creds' not found
kubectl get secret db-creds   # Verify it exists
```

7. Init Container Stuck

```
kubectl get pods
# NAME     READY   STATUS     — Init:0/1 means init container running/stuck
kubectl logs <pod> -c init-container-name
# Init containers often wait for services to be ready
```

Quick Diagnostic Reference

kubectl Command	What It Shows
kubectl describe pod <name>	Events, resource limits, probe status, node assignment
kubectl logs <name> --previous	Logs from last crashed container
kubectl get events --sort-by=.lastTimestamp	Cluster-wide recent events
kubectl top pods	Current CPU and memory usage
kubectl exec -it <name> -- sh	Shell inside running container for live diagnosis

> *Tip: kubectl describe pod and kubectl logs --previous are your two most important tools. Read the Events section at the bottom of describe output first — it almost always contains the exact reason for any pod failure.*

Q26. What is the command to access a pod and how can you define or create a Kubernetes object?

Answer

Accessing pods and creating Kubernetes objects are foundational daily operations. Knowing the full range of commands and approaches makes you efficient in debugging and managing workloads.

Accessing Pods — kubectl exec

```
# Interactive shell inside a running pod
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec -it <pod-name> -- /bin/sh     # For Alpine/minimal images

# Specific container in multi-container pod
kubectl exec -it <pod-name> -c <container-name> -- /bin/bash

# Run a single command without interactive shell
kubectl exec <pod-name> -- env
kubectl exec <pod-name> -- curl localhost:8080/health
kubectl exec <pod-name> -- cat /etc/app/config.yaml

# Port forward to access pod locally
kubectl port-forward pod/<pod-name> 8080:8080
kubectl port-forward deployment/<deploy-name> 8080:8080
kubectl port-forward service/<svc-name> 8080:80
```

Copying Files To/From Pods

```
# Copy from local to pod
kubectl cp ./local-file.txt <pod-name>:/tmp/file.txt

# Copy from pod to local
kubectl cp <pod-name>:/var/log/app.log ./app.log
```

Creating Kubernetes Objects

Method 1 — Imperative Commands (Quick, Development)

```
# Create a deployment
kubectl create deployment nginx --image=nginx:latest --replicas=3

# Expose as a service
kubectl expose deployment nginx --port=80 --type=LoadBalancer

# Create a ConfigMap
kubectl create configmap app-config \
```

```
  --from-literal=LOG_LEVEL=debug \
  --from-file=config.yaml

# Create a Secret
kubectl create secret generic db-creds \
  --from-literal=password='s3cr3t'

# Create a job
kubectl create job db-migrate \
  --image=myapp:latest \
  -- python manage.py migrate
```

Method 2 — YAML Manifests (Production Standard)

```
# Write the YAML manifest
cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: app
        image: myapp:v1.2
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
EOF
kubectl apply -f deployment.yaml
```

Method 3 — Generate YAML from Imperative Command

```
# Generate YAML without applying — useful for learning
kubectl create deployment my-app --image=myapp:v1 \
  --dry-run=client -o yaml > deployment.yaml

# Then edit and apply
kubectl apply -f deployment.yaml
```

apply vs create vs replace

Command	Behaviour
kubectl apply -f	Creates if not exists; updates if exists
kubectl create -f	Creates only; fails if already exists
kubectl replace -f	Replaces the entire object
kubectl patch	Partially updates a resource

> *Tip: Always use kubectl apply -f in CI/CD pipelines — it's idempotent. Store all manifests in Git. Never use kubectl create or kubectl run to make changes that aren't tracked in version control.*

Q27. Explain the folder structure of a basic Helm chart. What commands do you use to deploy with Helm?

Answer

Helm is the package manager for Kubernetes. It bundles related Kubernetes manifests into a 'chart' with templating support, versioning, and lifecycle management. Understanding Helm is essential for managing complex Kubernetes applications.

Helm Chart Directory Structure

```
my-app/                      # Chart directory (chart name)
├── Chart.yaml               # Chart metadata
├── values.yaml              # Default configuration values
├── values-dev.yaml          # Dev environment overrides
├── values-prod.yaml         # Prod environment overrides
├── charts/                  # Dependent charts (subcharts)
│   └── postgresql/          # e.g., bitnami/postgresql dependency
├── templates/               # Kubernetes manifest templates
│   ├── _helpers.tpl         # Template helpers and partials
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── hpa.yaml
│   ├── pdb.yaml
│   ├── serviceaccount.yaml
│   └── NOTES.txt            # Displayed after helm install
└── .helmignore              # Files to exclude from chart
```

Chart.yaml — Chart Metadata

```
apiVersion: v2
name: my-app
description: My application Helm chart
type: application
version: 1.3.0           # Chart version (semver)
appVersion: '2.5.1'      # Application version
dependencies:
- name: postgresql
  version: '13.x.x'
  repository: https://charts.bitnami.com/bitnami
  condition: postgresql.enabled
```

values.yaml — Templated Configuration

```
# values.yaml
replicaCount: 3

image:
  repository: myacr.azurecr.io/my-app
  tag: 'latest'
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: true
  host: api.myapp.com
  tls: true

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi

autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPUUtilizationPercentage: 70
```

Template File — deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-app.fullname" . }}
  labels:
```

```
    {{- include "my-app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: '{{ .Values.image.repository }}:{{ .Values.image.tag }}'
        ports:
        - containerPort: {{ .Values.service.port }}
        resources:
          {{- toYaml .Values.resources | nindent 10 }}
```

Essential Helm Commands

```
# Install a chart
helm install my-release ./my-app
helm install my-release ./my-app -f values-prod.yaml
helm install my-release ./my-app --set image.tag=v1.2.3

# Upgrade (update running release)
helm upgrade my-release ./my-app --set image.tag=v1.3.0
helm upgrade --install my-release ./my-app   # Install if not exists

# Rollback
helm history my-release
helm rollback my-release 2    # Roll back to revision 2

# Dry run — see what would be deployed
helm install my-release ./my-app --dry-run --debug

# Template — render manifests without deploying
helm template my-release ./my-app -f values-prod.yaml

# List, status, uninstall
helm list -n production
helm status my-release
helm uninstall my-release

# From registry
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/postgresql
helm install my-postgres bitnami/postgresql --version 13.2.0
```

Tip: Use helm upgrade --install in CI/CD pipelines — it handles both first deployment and subsequent updates with a single command. Combine with --wait --timeout 5m to block the pipeline until the rollout is healthy.

Q28. How would you implement fine-grained service discovery across 1000+ microservices using Envoy or Istio?

Answer

At 1000+ microservices, a service mesh is not optional — manual service discovery configuration doesn't scale. Istio (built on Envoy) provides automatic service discovery, traffic management, and security at this scale.

Istio Service Mesh Architecture

```
Data Plane: Envoy sidecar injected into every pod
  → intercepts all inbound/outbound traffic
  → enforces traffic rules, mTLS, observability

Control Plane: Istiod
  → watches Kubernetes API for Service/Endpoint changes
  → pushes xDS configuration to all Envoy sidecars
  → manages certificates for mTLS
```

Automatic Service Discovery

Istio's control plane (Istiod) uses Kubernetes service discovery — it watches Services and Endpoints and automatically programs all Envoy sidecars with routing information. No manual configuration per service pair is needed:

```
# Service A can reach Service B automatically
# Just reference by Kubernetes service name
curl http://payment-service.production.svc.cluster.local:8080/api/pay
# Envoy sidecar intercepts this, applies policies, routes with mTLS
```

Virtual Services — Fine-Grained Traffic Control

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: payment-routing
spec:
  hosts: [payment-service]
  http:
  # Route v2 users to new version
  - match:
    - headers:
        x-api-version:
          exact: v2
    route:
    - destination:
        host: payment-service
        subset: v2
  # All other traffic to v1
  - route:
    - destination:
        host: payment-service
        subset: v1
      weight: 95
    - destination:
        host: payment-service
        subset: v2
      weight: 5
    retries:
      attempts: 3
      perTryTimeout: 2s
    timeout: 10s
```

mTLS — Service Identity and Encryption

```
# Enable strict mTLS across entire mesh
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system
spec:
  mtls:
    mode: STRICT
# Every service-to-service call is encrypted and authenticated
# Services identify each other via SPIFFE certificates
```

AuthorizationPolicy — Zero-Trust Service Access

```
# Only allow payment-service to call fraud-service
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: fraud-service-policy
  namespace: production
spec:
  selector:
    matchLabels:
      app: fraud-service
  rules:
  - from:
    - source:
        principals:
        - cluster.local/ns/production/sa/payment-service
    to:
    - operation:
        methods: [POST]
        paths: [/api/fraud-check]
```

Observability with Istio

Istio automatically generates metrics, traces, and access logs for every service-to-service call — no application instrumentation needed:

- Metrics: Automatically exported to Prometheus (request rate, error rate, latency per service pair).
- Traces: Distributed traces via Jaeger/Zipkin for every cross-service call.
- Access logs: Every request logged with full metadata.

> *Tip: At 1000+ microservices, use Istio's DestinationRule with subsets and VirtualService for traffic management. But be conservative with the number of VirtualServices — each one adds configuration to push to all sidecars. Use wildcard host matching where possible to reduce control plane load.*

Q29. Explain how you'd leverage eBPF + Cilium to enforce network security policies at runtime, and what the advantages are over traditional CNIs?

Answer

eBPF (extended Berkeley Packet Filter) is a Linux kernel technology that allows running sandboxed programs in the kernel without changing kernel source code. Cilium uses eBPF to implement Kubernetes networking and security — with significant advantages over iptables-based CNIs.

Traditional CNI Limitations (iptables-based)

- Performance — iptables rule processing is O(n) — every packet is checked against every rule sequentially. At 10,000+ rules (common in large clusters), latency increases significantly.
- Visibility — iptables provides no application-layer visibility. You can only see IP/port, not HTTP methods, gRPC services, or DNS names.
- Complexity — Large iptables rule sets are hard to debug and verify.
- Conntrack limitations — Connection tracking tables have size limits and can be exhausted under high connection loads.

Cilium with eBPF — How It Works

Cilium replaces iptables with eBPF programs that run in the kernel. These programs are loaded dynamically, process packets in the kernel before they hit the network stack, and make decisions in microseconds:

```
# Install Cilium (replaces kube-proxy too)
helm repo add cilium https://helm.cilium.io/
helm install cilium cilium/cilium \
  --namespace kube-system \
  --set kubeProxyReplacement=strict \
  --set hubble.relay.enabled=true \
  --set hubble.ui.enabled=true
```

Advantages Over Traditional CNIs

Feature	iptables CNI
Rule processing	O(n) sequential scan
kube-proxy	Required
L7 policies	Not supported
Identity-based	IP address based
Observability	Limited
Encryption	External (IPsec overlay)

Cilium Network Policies — L7 Enforcement

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: api-to-payments
spec:
  endpointSelector:
    matchLabels:
      app: payment-service
  ingress:
  - fromEndpoints:
    - matchLabels:
        app: api-gateway
    toPorts:
    - ports:
      - port: '8080'
      rules:
        http:
        - method: POST              # Only POST requests
          path: /api/v1/payment     # Only to payment endpoint
          headers:
          - 'X-Auth-Token: .+'      # Must have auth header
```

Hubble — Network Flow Observability

```
# Install Hubble CLI
cilium hubble enable

# Observe all traffic
```

```
hubble observe --namespace production

# Observe dropped traffic
hubble observe --type drop --namespace production
# Shows exactly which NetworkPolicy dropped which flow

# Observe specific service
hubble observe --from-label app=api-gateway \
               --to-label app=payment-service
```

Transparent WireGuard Encryption

```
# Enable transparent pod-to-pod encryption
helm upgrade cilium cilium/cilium \
  --set encryption.enabled=true \
  --set encryption.type=wireguard
# All pod-to-pod traffic encrypted automatically
# No application changes needed
```

> *Tip: Cilium's identity-based security model is superior to IP-based NetworkPolicy for dynamic environments. Pods are identified by their labels (which don't change when pods are rescheduled and get new IPs), eliminating the race condition where IP-based policies briefly allow or deny wrong traffic during pod restarts.*

Q30. What happens when systemd units fail intermittently on EKS nodes? How do you detect and heal?

Answer

Intermittent systemd unit failures on EKS nodes cause hard-to-reproduce pod failures, node not-ready conditions, and kubelet failures. A systematic detection and auto-healing approach prevents these from becoming production incidents.

Critical systemd Units on EKS Nodes

Unit	Purpose
kubelet	Manages pods on the node
containerd	Container runtime
amazon-ssm-agent	SSM access and patching
amazon-cloudwatch-agent	Metrics and logs

Detection — Node Conditions

```
# Check node conditions
kubectl describe node <node-name> | grep -A20 Conditions
# Ready: False — kubelet not responding
# MemoryPressure: True — OOM conditions
# DiskPressure: True — disk full
# NetworkUnavailable: True — CNI issue

# Monitor node readiness in Prometheus
kube_node_status_condition{condition='Ready',status='true'} == 0
# Alert when this fires for > 2 minutes
```

Detection — systemd Journal Analysis

```
# SSH to node (or use SSM)
aws ssm start-session --target <instance-id>

# Check all failing units
systemctl --failed

# Check kubelet logs
journalctl -u kubelet -n 100 --no-pager
journalctl -u kubelet --since '30 minutes ago'

# Check for OOM kills
journalctl -k | grep -i 'oom\|killed'

# Check containerd
journalctl -u containerd -n 50
```

Auto-Healing — Automated Node Remediation

Use the Node Problem Detector + Cluster Autoscaler for automated detection and node replacement:

```
# Install Node Problem Detector
helm install node-problem-detector \
```

```
  deliveryhero/node-problem-detector \
  -n kube-system

# NPD detects: kubelet hang, container runtime hang,
# kernel deadlock, filesystem corruption, clock skew
# It sets Node Conditions that trigger node eviction
```

Cluster Autoscaler Node Auto-Repair

```
# EKS managed node groups have built-in health checks
# If a node fails EC2 health checks for > 10 minutes,
# AWS automatically terminates and replaces it

# Check node group health
aws eks describe-nodegroup \
  --cluster-name prod-cluster \
  --nodegroup-name prod-nodes \
  --query 'nodegroup.health'
```

Preventive Configuration

- Configure kubelet restart policy — systemd automatically restarts kubelet on failure:

```
# /etc/systemd/system/kubelet.service
[Service]
Restart=always
RestartSec=10
```

- Cordon and drain before termination — Node termination handler:

```
# Install AWS Node Termination Handler
helm install aws-node-termination-handler \
  eks/aws-node-termination-handler
# Gracefully drains nodes before spot interruptions or maintenance
```

- Monitor with CloudWatch Agent — ship systemd journal to CloudWatch:

```
# /opt/aws/amazon-cloudwatch-agent/etc/config.json
{"logs":{"logs_collected":{"files":{"collect_list":[
  {"file_path":"/var/log/journal/**","log_group_name":"/eks/nodes/journal"}
]}}}}
```

> *Tip: Configure PodDisruptionBudgets and pod topology spread constraints BEFORE a node failure occurs. When kubelet fails and the node goes NotReady, pods are evicted after 5 minutes (node.kubernetes.io/not-ready tolerations). PDBs ensure replacements are ready before the old node is fully drained.*

Q31. Walk through advanced kube-probe configurations to detect business logic failures, not just HTTP 200.

Answer

Standard readiness probes check if a service responds with HTTP 200. But a service can return 200 while failing business logic — the database connection pool is exhausted, the external payment API is timing out, or a background job is stuck. Advanced probe configurations detect these application-level failures.

The Problem with Basic HTTP 200 Probes

```
# Basic probe — INSUFFICIENT for complex apps
readinessProbe:
  httpGet:
    path: /health
    port: 8080
# Returns 200 even when:
# - DB connection pool is exhausted
# - External API is timing out
# - Cache is stale beyond acceptable threshold
# - Message processing is stuck
```

Advanced Health Endpoint — Aggregate Component Checks

Implement a /readiness endpoint that checks all critical dependencies and returns 503 if any are unhealthy:

```
# Python Flask example — comprehensive readiness check
@app.route('/readiness')
def readiness():
    checks = {}

    # Check database connection pool
    try:
        db.execute('SELECT 1', timeout=2)
        pool_stats = db.pool.status()
        if pool_stats.overflow > 5:  # Pool nearly exhausted
```

```
            checks['database'] = 'pool_pressure'
        else:
            checks['database'] = 'ok'
    except Exception as e:
        checks['database'] = f'error: {e}'

    # Check external payment API latency
    start = time.time()
    try:
        requests.get('https://payment-api/ping', timeout=1)
        latency = time.time() - start
        checks['payment_api'] = 'ok' if latency < 0.5 else 'slow'
    except:
        checks['payment_api'] = 'unreachable'

    # Check background job processor
    lag = redis.llen('processing_queue')
    checks['queue_lag'] = 'ok' if lag < 1000 else f'high:{lag}'

    is_healthy = all(v in ['ok'] for v in checks.values())
    status_code = 200 if is_healthy else 503
    return jsonify({'status': 'ok' if is_healthy else 'degraded',
                    'checks': checks}), status_code
```

Probe Configuration for Business Logic Detection

```
livenessProbe:
  httpGet:
    path: /health     # Simple — is process alive?
    port: 8080
  initialDelaySeconds: 30
  failureThreshold: 3
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /readiness  # Complex — all dependencies healthy?
    port: 8080
  initialDelaySeconds: 10
  failureThreshold: 2   # Remove from LB after 2 failures (20s)
  periodSeconds: 10
  successThreshold: 1

startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 30  # Allow 5 minutes startup
  periodSeconds: 10
```

gRPC Probes (Kubernetes 1.24+)

```
# For gRPC services — use standard gRPC health protocol
readinessProbe:
  grpc:
    port: 9090
    service: myapp.PaymentService
  periodSeconds: 10
```

Exec Probe — Custom Business Logic Scripts

```
readinessProbe:
  exec:
    command:
    - /bin/sh
    - -c
    - |
      # Check if payment processing is current
      LAST_PROCESSED=$(redis-cli get last_payment_timestamp)
      NOW=$(date +%s)
      AGE=$((NOW - LAST_PROCESSED))
      # Fail if no payment processed in 5 minutes during business hours
      if [ $AGE -gt 300 ]; then
        echo 'Payment processor stale'
        exit 1
      fi
      exit 0
  periodSeconds: 30
  failureThreshold: 2
```

httpHeaders — Checking Response Content

```
# Some health endpoints return 200 even when unhealthy
# Check a specific header or response body
readinessProbe:
  httpGet:
    path: /health
    port: 8080
    httpHeaders:
    - name: X-Health-Check
      value: kubernetes-probe  # Custom header to identify probe requests
```

> *Tip: Define separate health contracts for your services: /health returns 200 always (liveness — is the process running?); /readiness returns 503 when business-critical dependencies are unavailable (readiness — is the service ready to handle production traffic?). Document these contracts and never mix the two concerns.*

Q32. How do you handle DNS-level outages inside a service mesh without a full app redeploy?

Answer

DNS failures inside a Kubernetes service mesh cause service-to-service communication failures that appear as intermittent 502/503 errors or connection timeouts. Several techniques handle these without redeploying applications.

Understanding DNS in Kubernetes

```
Pod → CoreDNS → Kubernetes Service → Pod Endpoints
                  ↑
   DNS resolution at every step
   payment-service → payment-service.production.svc.cluster.local → 10.0.1.50
```

Problem 1 — CoreDNS Pods Overloaded or Crashing

```
# Check CoreDNS health
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns

# Check DNS resolution from a pod
kubectl exec -it test-pod -- nslookup payment-service.production
kubectl exec -it test-pod -- dig payment-service.production.svc.cluster.local

# Scale CoreDNS without redeployment
kubectl scale deployment coredns -n kube-system --replicas=5

# Check CoreDNS metrics
kubectl top pods -n kube-system -l k8s-app=kube-dns
```

Istio Solution — DNS Proxying

Istio can intercept DNS queries and resolve them via the Istio control plane, bypassing CoreDNS for service mesh traffic. Enable DNS proxying in Istio:

```
# Enable DNS proxying in meshconfig
kubectl edit configmap istio -n istio-system

# Add to meshConfig:
meshConfig:
  defaultConfig:
    proxyMetadata:
      ISTIO_META_DNS_CAPTURE: 'true'
      ISTIO_META_DNS_AUTO_ALLOCATE: 'true'

# Restart sidecar pods to pick up config
kubectl rollout restart deployment -n production
```

With DNS proxying, Istio's Envoy sidecar handles DNS resolution locally using the xDS service registry — CoreDNS is bypassed for known services. DNS outages affecting CoreDNS don't affect service mesh traffic.

ServiceEntry — Manual DNS Override

If an external service's DNS is failing, create a ServiceEntry to define the endpoint explicitly, bypassing DNS:

```
apiVersion: networking.istio.io/v1beta1
kind: ServiceEntry
metadata:
  name: external-payment-api
spec:
  hosts:
  - api.payment-provider.com
  location: MESH_EXTERNAL
```

```
  ports:
  - number: 443
    name: https
    protocol: TLS
  resolution: STATIC
  endpoints:
  - address: 203.0.113.50    # Direct IP — no DNS needed
    ports:
      https: 443
```

CoreDNS Configuration — Caching and Fallback

Configure CoreDNS with longer caching and fallback servers to handle transient DNS issues:

```
# Edit CoreDNS ConfigMap
kubectl edit configmap coredns -n kube-system

# Corefile with extended cache and fallback
.:53 {
    errors
    health
    cache 600 {      # Cache for 10 minutes — survive short outages
        success 9984
        denial 9984
    }
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        upstream
    }
    forward . 8.8.8.8 8.8.4.4 {  # Fallback DNS
        max_concurrent 1000
    }
    reload
}
```

Tip: Set CoreDNS HPA to automatically scale with cluster size. As a rule of thumb, run 1 CoreDNS replica per 500 pods, with a minimum of 2 replicas for HA. DNS is the cluster's nervous system — its performance directly affects all service communication.

Q33. A new Envoy config rollout silently broke mTLS between edge and mesh. What's your RCA trace?

Answer

Silent mTLS breaks are particularly dangerous — traffic may continue to flow but without encryption, or services may reject connections entirely. A systematic RCA using the service mesh's observability tools pinpoints the exact failure.

Step 1 — Confirm mTLS is Actually Broken

```
# Check Istio mTLS status between services
istioctl authn tls-check payment-service.production.svc.cluster.local
# Output shows: mTLS mode per service pair
# HOST                    STATUS    SERVER       CLIENT
# payment-service.prod    OK        STRICT       mTLS
# api-gateway.default     CONFLICT  STRICT       PERMISSIVE  ← mTLS broken

# Check if traffic is flowing without encryption
hubble observe --namespace production --type l7
# Look for: unencrypted connections that should be mTLS
```

Step 2 — Check the New Envoy Config

```
# When did the Envoy config change?
kubectl rollout history deployment/istio-ingressgateway -n istio-system

# What changed in the EnvoyFilter?
kubectl get envoyfilter -n istio-system -o yaml | \
  diff - <(git show HEAD~1:envoyfilter.yaml)

# Check applied EnvoyFilters
istioctl proxy-config all istio-ingressgateway-pod -n istio-system | grep -i tls
```

Step 3 — Inspect Envoy TLS Configuration

```
# Check what Envoy thinks its TLS configuration is
istioctl proxy-config listeners api-gateway-pod.production

# More detailed
kubectl exec -it api-gateway-pod -- \
  curl localhost:15000/config_dump | \
```

```
  python3 -m json.tool | grep -A20 'tls_context'
```

Step 4 — Check Certificate Validity

```
# Check if certificates are valid and not expired
istioctl proxy-config secret api-gateway-pod.production
# Should show: ROOTCA CERT CHAIN — Valid

# Check certificate details
kubectl exec -it api-gateway-pod -- \
  openssl s_client -connect payment-service:8080 -showcerts 2>/dev/null
```

Step 5 — Check PeerAuthentication and DestinationRule

```
# PeerAuthentication defines what MODE the server accepts
kubectl get peerauthentication -n production

# DestinationRule defines what the CLIENT sends
kubectl get destinationrule -n production -o yaml | grep -A5 'tls:'

# CONFLICT happens when:
# PeerAuthentication: mode: STRICT (requires mTLS)
# DestinationRule: mode: DISABLE or SIMPLE (client sends plain or TLS, not mTLS)
```

Common Root Causes

Root Cause	Detection
DestinationRule missing TLS mode	istioctl authn tls-check shows CONFLICT
EnvoyFilter overriding TLS context	Config dump shows modified TLS context
Namespace missing istio-injection label	kubectl get namespace shows no injection label
Certificate rotation failure	istioctl proxy-config secret shows EXPIRED

Immediate Remediation

```
# Revert the Envoy config change
kubectl rollout undo deployment/istio-ingressgateway -n istio-system

# Verify mTLS restored
istioctl authn tls-check payment-service.production.svc.cluster.local
```

> *Tip: After every Istio/Envoy configuration change, run istioctl analyze to check for configuration conflicts and validate the mesh configuration. Many mTLS breaks are caught by this analysis before they reach production.*

Q34. HPA refuses to scale even though Prometheus shows CPU > 80%. Diagnose with cloud + K8s metrics.

Answer

This is a comprehensive follow-up to Q9. When HPA doesn't scale despite high CPU, the issue is often in the metrics pipeline between Prometheus and HPA, or in HPA's own configuration.

The Metrics Pipeline

```
Prometheus (collects metrics)
  → Prometheus Adapter (exposes as Kubernetes custom metrics API)
  → HPA reads from custom.metrics.k8s.io or metrics.k8s.io
  → HPA calculates: actual / target = scale factor
```

Step 1 — Verify HPA is Reading Metrics

```
kubectl get hpa my-app-hpa -o yaml | grep -A20 'status'
# status.currentMetrics should show the current value
# If it shows <unknown>: HPA cannot read the metric

# Check the exact metric name HPA is using
kubectl describe hpa my-app-hpa | grep -A5 'Metrics'
# Metrics: 90% / 70% — HPA CAN see the metric but something else prevents scaling
```

Step 2 — Prometheus Shows 80% but HPA Sees Different

The Prometheus Adapter may be computing the metric differently than Prometheus UI shows. Verify what HPA actually reads:

```
# Query the custom metrics API directly
kubectl get --raw '/apis/custom.metrics.k8s.io/v1beta1/namespaces/default/pods/*/cpu_usage_pct'

# Or check the metrics API for standard CPU
kubectl get --raw '/apis/metrics.k8s.io/v1beta1/namespaces/default/pods' | jq .

# Compare: what Prometheus shows vs what metrics API returns
# Prometheus: avg CPU per pod = 80%
# Metrics API: CPU usage reported differently (might be raw cores, not %)
```

Step 3 — Check Prometheus Adapter Configuration

```
kubectl get configmap adapter-config -n monitoring -o yaml
# Verify the metric name matches what HPA references
# seriesQuery matches the right Prometheus metric
# metricsQuery correctly computes per-pod average
```

Step 4 — Cloud Metrics Mismatch (EKS/AKS)

```
# EKS: Container Insights may report differently from metrics-server
aws cloudwatch get-metric-statistics \
  --namespace ContainerInsights \
  --metric-name pod_cpu_utilization \
  --dimensions Name=PodName,Value=my-app-xyz \
  --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 60 --statistics Average

# Compare CloudWatch reported CPU vs metrics-server CPU
kubectl top pods my-app-xyz
# If they differ significantly, metrics-server may have stale data
```

Step 5 — HPA Status Conditions

```
kubectl describe hpa | grep -A30 'Conditions'
# AbleToScale=False: permission issue or target not found
# ScalingActive=False: disabled — check why
# ScalingLimited=True: at maxReplicas or due to behavior limits
# reason: DesiredWithinRange: CPU is 80% but target is 70%
#  — this SHOULD be scaling. If not, check ScalingLimited
```

Step 6 — Replica Count at Maximum

```
kubectl get hpa
# REPLICAS=20 MAXPODS=20 — at maximum. HPA cannot scale further
# Fix: increase maxReplicas
kubectl patch hpa my-app-hpa -p '{"spec":{"maxReplicas":50}}'
```

> *Tip: When HPA shows the right metric value but still doesn't scale, check ScalingLimited=True in the conditions. It means HPA calculated a higher replica count but was constrained by maxReplicas or behavior policies. Increasing maxReplicas or adjusting behavior policies usually resolves this.*

Q35. You introduced a sidecar-based caching layer. Suddenly, tail latency spikes. What's your debug path?

Answer

A sidecar-based caching layer adds a proxy process to every pod. When it causes tail latency spikes (p99/p999 much higher than p50), the culprit is usually contention, resource starvation, or serialisation in the sidecar.

Understanding the Architecture

```
Incoming request → App container → Sidecar cache → External service
                                        ↓
                                 Cache hit: return cached
                                 Cache miss: forward to external, cache, return
```

Step 1 — Confirm Sidecar is the Cause

```
# Compare latency before and after sidecar introduction
# Prometheus query — tail latency per container
histogram_quantile(0.99, rate(http_duration_seconds_bucket{container='app'}[5m]))
histogram_quantile(0.99, rate(http_duration_seconds_bucket{container='cache-sidecar'}[5m]))

# Check if sidecar's p99 is high
# If sidecar latency is high → sidecar is the bottleneck
# If app latency is high but sidecar is fast → sidecar is causing app resource pressure
```

Step 2 — Check Resource Contention

```
# Is the sidecar starving the main app?
```

```
kubectl top pods --containers
# NAME    CONTAINER       CPU    MEMORY
# my-pod  app             450m   200Mi
# my-pod  cache-sidecar   490m   800Mi  ← sidecar using too much

# Check if sidecar is CPU throttled
kubectl exec <pod> -c cache-sidecar -- cat /sys/fs/cgroup/cpu/cpu.stat
# nr_throttled: high number indicates throttling

# Fix: set resource limits on sidecar
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

Step 3 — Check for Lock Contention or Serialisation

```
# Check if sidecar has a single-threaded bottleneck
kubectl exec <pod> -c cache-sidecar -- top
# If 1 vCPU at 100% but others idle: single-threaded bottleneck

# Check goroutine/thread count (Go-based sidecar)
kubectl exec <pod> -c cache-sidecar -- \
  curl localhost:6060/debug/pprof/goroutine?debug=1 | head -20
```

Step 4 — Network Overhead Between Containers

```
# Measure loopback latency between app and sidecar
kubectl exec <pod> -c app -- \
  curl -w '%{time_total}' http://localhost:3000/cache/test

# p99 latency to sidecar should be < 1ms on loopback
# If > 5ms: connection pooling issue in the app→sidecar client
```

Step 5 — Cache Hit Rate

```
# Low cache hit rate means most requests miss and go to external service
# Double the latency of every request (app + cache miss + external)
kubectl exec <pod> -c cache-sidecar -- \
  curl localhost:8080/metrics | grep cache_hit_rate
# cache_hit_rate 0.3  ← only 30% hitting cache — too low

# Fix: increase cache size, adjust TTL, or review cache key design
```

Step 6 — GC Pauses in the Sidecar

```
# If sidecar is JVM or Go-based, GC pauses cause tail latency
kubectl logs <pod> -c cache-sidecar | grep -i 'gc\|pause'

# Go pprof heap analysis
kubectl exec <pod> -c cache-sidecar -- \
  curl localhost:6060/debug/pprof/heap > heap.prof
go tool pprof heap.prof
```

> *Tip: The most common cause of sidecar-induced tail latency is CPU throttling — the sidecar is resource-limited and stalls on request spikes. Set generous CPU limits (or remove them for latency-sensitive sidecars on dedicated nodes) and monitor throttling with container_cpu_cfs_throttled_periods_total.*

Q36. 504 errors on the playback service. Cloud LB shows healthy, mesh sidecars pass, but users can't stream. Triage?

Answer

504 Gateway Timeout when the LB reports healthy and sidecars pass means the timeout is happening deeper in the request chain. Systematic layer-by-layer triage finds where the 30-second (or configured) timeout is being hit.

Request Path — Where Can the 504 Originate?

```
User → CDN → Cloud LB → Istio Ingress Gateway → Envoy Sidecar → App → Backend
                              ↑ each hop can timeout
```

Step 1 — Identify Where the 504 is Generated

```
# Check Istio access logs — what does the ingressgateway report?
kubectl logs -n istio-system deployment/istio-ingressgateway | \
  grep '504\|timeout' | tail -20
```

```
# Check application pods
kubectl logs -l app=playback-service | grep -i 'timeout\|504' | tail -20

# Check Envoy upstream timeout
istioctl proxy-config cluster playback-pod.production | grep -i timeout
```

Step 2 — Check Upstream Timeout Configuration

```
# VirtualService may have too-short timeout
kubectl get virtualservice playback-vs -o yaml | grep timeout
# timeout: 10s  ← if playback takes > 10s to start, this causes 504

# For video streaming — playback init can take > 10s
# Fix: increase timeout for playback service
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
spec:
  http:
  - route:
    - destination:
        host: playback-service
    timeout: 30s   # Increase for streaming
```

Step 3 — Check Backend Service Latency

```
# Is playback-service calling a slow backend?
# Use Jaeger/Tempo traces to find the slow span
kubectl port-forward -n monitoring svc/jaeger-query 16686:16686
# Open browser: http://localhost:16686
# Search: service=playback-service, duration > 10s
# Trace shows: playback → cdn-auth-service → 25s (!!!)
```

Step 4 — Check CDN and Video Segment URLs

For streaming specifically, 504 can occur in the CDN/video delivery layer that isn't directly visible in Kubernetes:

```
# Check if CDN is timing out fetching segments from origin
curl -v https://cdn.company.com/playback/video-segment-001.ts
# Check response time and CDN headers
# X-Cache: MISS — CDN fetching from origin — origin may be slow
```

Step 5 — Check DRM / Token Validation Service

```
# Streaming often requires DRM token validation — this is a common bottleneck
kubectl top pods -l app=drm-service
kubectl logs -l app=drm-service | grep -i 'slow\|timeout'

# Check if DRM service has enough replicas
kubectl get hpa drm-service-hpa
```

Step 6 — Check Resource Limits on Playback Pods

```
kubectl top pods -l app=playback-service
# If CPU near limit: throttling causes slow response → timeout

kubectl exec playback-pod -- cat /sys/fs/cgroup/cpu/cpu.stat | grep throttled
```

Resolution Summary

Cause	Fix
VirtualService timeout too short	Increase timeout for streaming use case
Slow DRM/auth backend	Scale DRM service, add caching, increase timeout
CDN fetching from slow origin	Configure CDN origin timeout, preload popular content
CPU throttling on playback pods	Increase CPU limits or remove limits on dedicated nodes
Database query under load	Add read replicas, implement caching for playback metadata

> *Tip: For streaming services, implement client-side retry with exponential backoff. A single 504 shouldn't interrupt playback — the player can retry the segment fetch. Combine this with server-side fixes to prevent 504s cascading into user-visible playback interruptions.*

Q37. How would you approach debugging a pod that's in a CrashLoopBackOff state?

Answer

The existing answer from the source document is correct. Expanding with complete diagnostic steps and all common root causes.

Understanding CrashLoopBackOff

CrashLoopBackOff means the container starts, crashes (exits with non-zero code), and Kubernetes restarts it. After repeated crashes, Kubernetes adds increasing delays between restarts (10s, 20s, 40s, up to 5 minutes). The 'BackOff' is the delay mechanism.

Step 1 — Get the Exit Code and Reason

```
kubectl describe pod <pod-name>
# Last State:
#   Terminated:
#     Reason: Error
#     Exit Code: 1      ← application error
#     Exit Code: 137    ← OOMKilled (128 + 9)
#     Exit Code: 143    ← SIGTERM (graceful shutdown requested)
#     Reason: OOMKilled ← out of memory
```

Step 2 — Read the Logs

```
# Current container logs
kubectl logs <pod-name>
# Logs from PREVIOUS crashed container — most useful
kubectl logs <pod-name> --previous

# Multi-container pod — specify container
kubectl logs <pod-name> -c <container-name> --previous
```

Common Root Causes and Fixes

Exit Code / Reason	Root Cause
1 (generic error)	Application startup exception
137 / OOMKilled	Memory limit exceeded
1 with 'can't open config'	Missing ConfigMap or Secret
0 (exits cleanly)	App exits after task (wrong for a server)
139 (SIGSEGV)	Segmentation fault — app crash
Exit, readiness fails	App starts but fails readiness probe

Step 3 — Debug Without Restarting

```
# Run interactive shell to inspect environment
kubectl debug -it <pod-name> --image=busybox --share-processes

# Change command to keep container running for inspection
kubectl edit deployment my-app
# Change: command: ['/bin/sleep', '3600']  # Override crash-looping command
# Then exec in and manually run the app to see the error
kubectl exec -it <pod-name> -- /app/run.sh
```

> *Tip: The exit code is the fastest clue: 137=OOM (increase memory), 1=application error (read logs), 0=app exits normally (wrong CMD). Never restart without reading --previous logs first — the answer is always in the last crash's output.*

Q38. Can you explain the difference between a DaemonSet and a Deployment?

Answer

The existing answer is correct and complete. Expanding with use cases, examples, and key operational differences.

Deployment — For Replicated, Schedulable Workloads

A Deployment maintains N replicas of a pod, scheduled on any available node. Pods are interchangeable — any pod can handle any request. The scheduler places pods optimally across nodes.

```
# Deployment: 3 replicas, placed on any nodes
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
```

```
  # 3 pods running on whichever 3 nodes have capacity
  # If a pod dies, scheduler places replacement anywhere
```

DaemonSet — One Pod Per Node

A DaemonSet ensures exactly one pod runs on every node (or every node matching a selector). When new nodes join the cluster, DaemonSet pods are automatically scheduled on them. When nodes leave, their pods are garbage collected.

```
# DaemonSet: one pod per node — on ALL nodes
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
spec:
  selector:
    matchLabels:
      app: log-collector
  template:
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.1
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath:
          path: /var/log    # Access node's log directory
```

Key Differences

Aspect	Deployment
Pod count	Configurable (replicas: N)
Scheduling	Scheduler places optimally
Scaling	kubectl scale or HPA
Use case	Stateless apps, APIs, workers
hostPath access	Unusual
Resource consumption	Fixed count

Common DaemonSet Use Cases

- Log collection agents (Fluent Bit, Fluentd) — must run on every node to collect all container logs from /var/log/containers.
- Metrics agents (node-exporter, Datadog agent, CloudWatch Agent) — collect node-level metrics.
- Network plugins (Cilium, Calico, Weave) — CNI runs on every node for pod networking.
- Security agents (Falco, Sysdig, CrowdStrike) — runtime security monitoring of every node.
- Storage drivers (EBS CSI, NFS provisioner) — must be on every node where pods with PVCs might schedule.

> *Tip: DaemonSets can be configured with nodeSelector or affinity to run only on specific nodes (e.g., only on GPU nodes, only on worker nodes not control plane). Use this to run GPU drivers only on GPU nodes or deploy monitoring agents only on non-system nodes.*

Q39. How do you handle persistent storage in Kubernetes?

Answer

The existing answer is correct. This answer expands with complete implementation details, StorageClass configuration, and production considerations.

The Storage Abstraction Layers

```
Physical Storage (EBS, NFS, Azure Disk, etc.)
    ↓
PersistentVolume (PV) — cluster resource representing physical storage
    ↓
StorageClass — defines how PVs are dynamically provisioned
    ↓
PersistentVolumeClaim (PVC) — namespace-scoped request for storage
    ↓
Pod — mounts PVC as a volume
```

Dynamic Provisioning with StorageClass

Modern Kubernetes uses dynamic provisioning — a PVC creates a PV automatically via the StorageClass provisioner:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: 'true'
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer  # Create in same AZ as pod
reclaimPolicy: Retain                    # Keep volume when PVC deleted
allowVolumeExpansion: true
parameters:
  type: gp3
  iops: '3000'
  throughput: '125'
  encrypted: 'true'
```

PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
  namespace: production
spec:
  accessModes: [ReadWriteOnce]    # One node can read+write at a time
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 50Gi
```

Pod Using PVC

```
spec:
  containers:
  - name: app
    image: myapp:v1
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: app-data
```

Access Modes

Access Mode	Abbreviation
ReadWriteOnce	RWO
ReadOnlyMany	ROX
ReadWriteMany	RWX
ReadWriteOncePod	RWOP

Expanding a PVC Online

```
# Edit PVC to request more storage
kubectl patch pvc app-data -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'
# EBS CSI driver expands the volume online — no pod restart needed

# Verify expansion
kubectl get pvc app-data
# CAPACITY shows new size after expansion completes
```

> *Tip: Use ReadWriteMany (EFS on AWS, Azure Files on Azure) when multiple pods need shared storage — for example, a web server cluster sharing uploaded files. ReadWriteOnce (EBS, Azure Disk) is sufficient and cheaper for databases and single-pod workloads.*

Q40. What strategies would you use to optimize resource utilization in a Kubernetes cluster?

Answer

Resource optimisation in Kubernetes is a continuous process across four dimensions: right-sizing pods, intelligent scheduling, efficient node usage, and autoscaling. The existing answer is correct — this expands with implementation details.

1. Resource Requests and Limits — The Foundation

```
resources:
  requests:
    cpu: 100m       # Guaranteed CPU — used for scheduling
    memory: 128Mi   # Guaranteed memory — used for scheduling
  limits:
    cpu: 500m       # Maximum CPU — throttled if exceeded
    memory: 256Mi   # Maximum memory — OOMKilled if exceeded
```

Set requests based on p50 usage and limits at p99 usage. Requests too high wastes node capacity; limits too low causes OOMKill or throttling.

2. Vertical Pod Autoscaler (VPA) — Right-Sizing

```
# VPA recommends or auto-adjusts resource requests
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: 'Off'     # Recommendation only — review before applying

# Check recommendations
kubectl describe vpa my-app
# Lower Bound: 50m/64Mi  Target: 200m/256Mi  Upper Bound: 1000m/1Gi
```

3. LimitRange — Enforce Defaults Namespace-Wide

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: production
spec:
  limits:
  - type: Container
    default:
      cpu: 500m
      memory: 256Mi
    defaultRequest:
      cpu: 100m
      memory: 128Mi
    max:
      cpu: '4'
      memory: 4Gi
```

4. HPA — Horizontal Pod Autoscaler

Scale pods out during peak and in during quiet periods — don't run peak capacity 24/7:

```
minReplicas: 2     # Never below 2 (HA)
maxReplicas: 20    # Scale to 20 at peak
# At night: 2 replicas. During day: 10-15 replicas. Cost savings: ~60%
```

5. Karpenter — Efficient Node Provisioning

Karpenter provisions the smallest nodes that fit pending pods — no wasted capacity. It bin-packs workloads and removes empty nodes in 30 seconds:

```
# Karpenter chooses the right instance type automatically
# If pending pods need: 2 vCPU, 4Gi — Karpenter picks t3.medium
# Rather than launching m5.xlarge (8 vCPU, 16Gi) and wasting 75%
```

6. Pod Priority and QoS Classes

```
# Guaranteed QoS: requests == limits → scheduled first, evicted last
# Burstable QoS: requests < limits → normal scheduling
# BestEffort QoS: no requests/limits → evicted first during pressure

# Set priority for critical services
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
---
spec:
  priorityClassName: high-priority
```

> *Tip: Run kubectl resource-capacity (plugin) or Goldilocks (VPA in recommendation mode) to identify over-provisioned and under-provisioned deployments. Target average CPU utilisation of 60-70% across nodes — below 40% indicates over-provisioning, above 80% indicates under-provisioning risk.*

Q41. How would you implement a blue-green deployment in Kubernetes? (detailed)

Answer

This is a duplicate of Q4 and Q17 — covered comprehensively there. The existing answer in the source document is: Create two identical deployments (Blue and Green) with different labels. Use a service with a selector matching the blue deployment's labels. Update the green deployment with the new version. Test the green deployment to ensure it's working correctly.

This answer is correct. For the complete implementation with YAML examples, automation pipeline, Helm approach, and rollback — see Q4 (Blue-Green Deployment) and Q17 (Fully Automated Blue-Green) in this document.

Quick Reference — Key Commands

```
# Deploy green
kubectl set image deployment/my-app-green app=myapp:v2.0
kubectl rollout status deployment/my-app-green

# Switch traffic
kubectl patch service my-app -p '{"spec":{"selector":{"version":"green"}}}'

# Rollback
kubectl patch service my-app -p '{"spec":{"selector":{"version":"blue"}}}'
```

Q42. How would you implement auto-scaling in Kubernetes based on custom metrics?

Answer

The existing answer is correct — it identifies the two key steps. This answer provides complete implementation details.

Architecture

```
Application → Prometheus (collects custom metrics)
           → Prometheus Adapter (exposes as K8s custom metrics API)
           → HPA reads custom.metrics.k8s.io
           → HPA scales deployment
```

Step 1 — Application Exposes Custom Metrics

```
# Python Flask with prometheus_client
from prometheus_client import Counter, Histogram, generate_latest

REQUEST_RATE = Counter('http_requests_total', 'Total requests', ['method', 'status'])
QUEUE_DEPTH = Counter('queue_messages_pending', 'Messages in queue')

@app.route('/metrics')
def metrics():
    return generate_latest()
```

Step 2 — ServiceMonitor for Prometheus

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
spec:
  endpoints:
  - port: metrics
    path: /metrics
    interval: 15s
```

Step 3 — Prometheus Adapter Configuration

```
# prometheus-adapter values.yaml
rules:
  custom:
  - seriesQuery: 'http_requests_total{namespace!="",pod!=""}'
    resources:
      overrides:
        namespace: { resource: namespace }
        pod: { resource: pod }
    name:
      matches: '^(.*)_total'
```

```
      as: '${1}_per_second'
    metricsQuery: 'rate(<<.Series>>{<<.LabelMatchers>>}[2m])'
```

Step 4 — HPA with Custom Metric

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: my-service
  minReplicas: 2
  maxReplicas: 50
  metrics:
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: '100'   # Scale when >100 req/s per pod

# Verify custom metric is available
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | jq '.resources[].name'
```

Tip: Use KEDA (Kubernetes Event-Driven Autoscaling) instead of the Prometheus Adapter for simpler custom metric scaling. KEDA has built-in scalers for Prometheus, SQS, Kafka, Redis, and 50+ other sources — no adapter configuration needed.

Q43. What's your strategy for managing container image security across all stages of a DevOps pipeline?

Answer

Container image security is a multi-stage concern — shift security left by checking at every stage from code to runtime. A single check at one stage is insufficient.

Stage 1 — Developer (Pre-Commit)

- Dockerfile linting — Use hadolint to catch insecure Dockerfile practices:

```
hadolint Dockerfile
# DL3007: Using latest is prone to errors if image upgrades
# DL3008: Pin versions in apt-get install
```

- Secret scanning — detect-secrets or TruffleHog pre-commit hooks prevent credential exposure.

Stage 2 — CI Build

```
# Scan image for CVEs with Trivy
trivy image \
  --exit-code 1 \
  --severity CRITICAL,HIGH \
  --no-progress \
  --format sarif \
  --output results.sarif \
  my-app:${{ github.sha }}

# Also scan IaC and Kubernetes manifests
trivy config ./k8s/
```

Stage 3 — Container Registry (ACR/ECR)

- Amazon ECR — Enable enhanced scanning with Amazon Inspector. Automatically scans images on push and continuously monitors for new CVEs.
- Azure ACR — Enable Defender for Containers for continuous scanning.
- Image signing — Sign images with cosign (Sigstore) to ensure only trusted images run:

```
# Sign the image
cosign sign --key cosign.key myacr.io/my-app:$SHA

# Verify before deploying
cosign verify --key cosign.pub myacr.io/my-app:$SHA
```

Stage 4 — Kubernetes Admission Control

Block unsigned or vulnerable images from being deployed using admission controllers:

```
# Kyverno policy – require signed images
apiVersion: kyverno.io/v1
kind: ClusterPolicy
spec:
  rules:
```

```
  - name: check-image-signature
    match:
      resources:
        kinds: [Pod]
    verifyImages:
    - imageReferences: ['myacr.io/*']
      attestors:
      - entries:
        - keyless:
            subject: 'https://github.com/company/repo/.github/workflows/*'
```

Stage 5 — Runtime (Falco)

Detect runtime threats — see Q23 for full Falco implementation. Key rules for image security:

- Alert when a container runs an unexpected binary (drift detection).
- Alert when a container downloads and executes a binary at runtime.
- Alert when a container runs as root unexpectedly.

Stage	Tool
Developer	hadolint, detect-secrets
CI Build	Trivy, Grype
Registry	ECR/ACR scanning, cosign
Admission	Kyverno, OPA Gatekeeper
Runtime	Falco, Sysdig

> *Tip: Use a vulnerability severity threshold that evolves over time: start by blocking CRITICAL CVEs, then extend to HIGH after teams have processes to handle them. An alert on every MEDIUM CVE creates noise and alert fatigue — prioritise based on reachability and exploitability.*

Q44. How do you dynamically provision ephemeral environments (dev/test) using pipelines?

Answer

Ephemeral environments are short-lived, automatically created for each PR or feature branch, and destroyed when no longer needed. They enable parallel development, isolated testing, and cost control.

Architecture — Per-PR Namespaces

Create a dedicated Kubernetes namespace per pull request. The namespace contains all services needed for testing. It's destroyed when the PR is closed.

```
# Pipeline creates namespace per PR
NAMESPACE=pr-${{ github.event.number }}
kubectl create namespace $NAMESPACE

# Label with TTL for automatic cleanup
kubectl label namespace $NAMESPACE \
  pr=${{ github.event.number }} \
  created-at=$(date +%s) \
  ttl=24h
```

Automated Namespace Provisioning

```
# .github/workflows/ephemeral-env.yml
name: Ephemeral Environment
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  provision:
    steps:
    - name: Deploy to ephemeral namespace
      run: |
        NAMESPACE=pr-${{ github.event.number }}
        SUBDOMAIN=pr-${{ github.event.number }}

        # Create namespace
        kubectl create namespace $NAMESPACE --dry-run=client -o yaml | kubectl apply -f -
```

```
        # Deploy with Helm
        helm upgrade --install my-app ./charts/my-app \
          --namespace $NAMESPACE \
          --set image.tag=${{ github.sha }} \
          --set ingress.host=${SUBDOMAIN}.preview.company.com \
          --set resources.requests.cpu=50m \
          --set resources.requests.memory=64Mi \
          --set replicaCount=1 \
          --wait --timeout=5m

        # Comment PR with URL
        gh pr comment ${{ github.event.number }} \
          --body '🚀 Preview: https://${SUBDOMAIN}.preview.company.com'
```

Dynamic DNS with External-DNS

Automatically create DNS records for ephemeral environments using External-DNS:

```
# External-DNS watches Ingress resources and creates Route53/Azure DNS records
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    external-dns.alpha.kubernetes.io/hostname: pr-123.preview.company.com
spec:
  rules:
  - host: pr-123.preview.company.com
    # External-DNS creates DNS record automatically
```

Automatic Cleanup

```
# Cleanup job when PR is closed
on:
  pull_request:
    types: [closed]

  cleanup:
    steps:
    - run: |
        kubectl delete namespace pr-${{ github.event.number }}
        # External-DNS automatically removes the DNS record
```

Namespace TTL Controller

```
# Install Namespace Terminator — auto-deletes namespaces after TTL
helm install namespace-terminator \
  helm-charts/namespace-terminator

# Any namespace with label 'ttl=24h' is deleted after 24 hours
# Catches orphaned namespaces if the PR cleanup workflow fails
```

> *Tip: Use Telepresence or Skaffold for inner-loop development — a developer's local code runs as if it's inside the Kubernetes cluster, with fast rebuild cycles. This reduces the need for a full ephemeral environment for every code change while developing.*

Q45. In a monorepo setup, how do you ensure that only relevant services are built and deployed in a CI/CD pipeline?

Answer

In a monorepo with many services, rebuilding and redeploying everything on every commit wastes build time, increases risk, and slows delivery. Smart change detection ensures only affected services are built and deployed.

The Core Problem

```
monorepo/
├── services/
│   ├── auth-service/
│   ├── payment-service/
│   ├── order-service/
│   └── notification-service/
├── shared-libs/
│   └── common-utils/
└── infra/
    └── terraform/
```

If only auth-service/src/login.py changes, we should build and deploy ONLY auth-service — not payment-service, order-service, and notification-service.

Approach 1 — Git Diff-Based Detection

```
# Detect changed services using git diff
CHANGED_SERVICES=$(git diff --name-only HEAD~1 HEAD \
  | grep '^services/' \
  | cut -d/ -f2 \
  | sort -u)

echo "Changed services: $CHANGED_SERVICES"
# Output: auth-service

# Build and deploy only changed services
for SERVICE in $CHANGED_SERVICES; do
  echo "Building $SERVICE"
  docker build -t myapp/$SERVICE:$SHA ./services/$SERVICE/
  helm upgrade --install $SERVICE ./charts/$SERVICE \
    --set image.tag=$SHA
done
```

Approach 2 — GitHub Actions Path Filters

```
# Separate workflow per service with path triggers
name: Auth Service
on:
  push:
    paths:
    - 'services/auth-service/**'
    - 'shared-libs/**'      # Rebuild if shared libs change
    - '.github/workflows/auth-service.yml'

jobs:
  build-and-deploy:
    steps:
    - name: Build auth-service
      run: docker build -t auth-service:${{ github.sha }} services/auth-service/
```

Approach 3 — Nx or Turborepo (Build Orchestration)

Build orchestration tools understand service dependencies and calculate what needs rebuilding:

```
# Nx — detects affected projects
npx nx affected:build --base=HEAD~1 --head=HEAD
npx nx affected:deploy --base=HEAD~1 --head=HEAD
# If shared-utils changed: rebuilds ALL services that depend on it
# If only auth-service changed: rebuilds only auth-service
```

Handling Shared Library Changes

When a shared library changes, all services that depend on it must be rebuilt:

```
# Build dependency graph
# shared-libs/common-utils/ → auth-service, payment-service
# If common-utils changes: build auth-service AND payment-service

# Implement in shell
if git diff --name-only HEAD~1 HEAD | grep -q 'shared-libs/'; then
  echo 'Shared lib changed — building ALL services'
  SERVICES_TO_BUILD=$(ls services/)
else
  SERVICES_TO_BUILD=$(git diff --name-only HEAD~1 HEAD \
    | grep '^services/' | cut -d/ -f2 | sort -u)
fi
```

Caching — Make Rebuilds Fast

```
# Use Docker build cache keyed to service directory
docker build \
  --cache-from myregistry.io/auth-service:cache \
  --build-arg BUILDKIT_INLINE_CACHE=1 \
  -t myregistry.io/auth-service:$SHA \
  -t myregistry.io/auth-service:cache \
  services/auth-service/
```

> *Tip: Use a dependency mapping file (deps.json) that explicitly declares which services depend on which shared libraries. Check this file in the change detection step to ensure rebuilds cascade correctly when shared code changes — more reliable than purely inferring dependencies from import statements.*
